

Received 24 March 2025, accepted 11 April 2025, date of publication 16 April 2025, date of current version 30 April 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3561375

RESEARCH ARTICLE

Supervised Anomaly Detection in Univariate Time-Series Using 1D Convolutional Siamese Networks

AYAN CHATTERJEE^{1,2}, (Member, IEEE), VAJIRA THAMBAWITA³, (Member, IEEE),
MICHAEL A. RIEGLER^{2,3}, (Member, IEEE), AND PÅL HALVORSEN^{2,3}, (Member, IEEE)

¹Department of Digital Technology, STIFTELSEN NILU, 2007 Kjeller, Norway

²Department of Information Technology, Oslo Metropolitan University, 0166 Oslo, Norway

³Department of Holistic Systems, Simula Metropolitan Center for Digital Engineering (SimulaMet), 0170 Oslo, Norway

Corresponding author: Pål Halvorsen (pal.halvorsen@oslomet.no)

This work was supported by the Simula Metropolitan Center for Digital Engineering (SimulaMet), the University of Agder (UiA), NILU, and OsloMet.

ABSTRACT In time-series data analysis, identifying anomalies is crucial for maintaining data integrity and ensuring accurate analyses and decision-making. Anomalies can compromise data quality and operational efficiency. The complexity of time-series data, with its temporal dependencies and potential non-stationarity, makes anomaly detection challenging but essential. Our research introduces ADSiamNet, a 1D Convolutional Neural Network-based Siamese network model for anomaly detection and rectification. ADSiamNet effectively identifies localized patterns in time-series data and smooths detected anomalies using a quantile-based technique. In tests with physical activity data from Actigraph watches and MOX2-5 sensors, ADSiamNet achieved accuracies of 98.65% and 85.0%, respectively, outperforming other supervised anomaly detection methods. The model uses a contrastive loss function to compare input sequences and adjusts network weights iteratively during training to recognize intricate patterns. Additionally, we evaluated various univariate time-series forecasting algorithms on datasets with and without anomalies. Results show that anomaly-smoothed data reduces forecasting errors, highlighting our approach's effectiveness in enhancing time-series data analysis's integrity and reliability. Future research will focus on multivariate time-series datasets.

INDEX TERMS Univariate time-series, anomaly detection, siamese neural networks, 1D CNNs, quantile smoothing, predictive modeling.

I. INTRODUCTION

Time-series data, represented as chronological sequences of data points [1], [2], serves as the cornerstone of many fields such as finance [3], economics [4], health [5], engineering [6], and social science [7]. Anomalies in time-series data can lead to false health indicators, misdiagnosis, and ineffective interventions, especially in critical areas such as physical activity monitoring and healthcare applications. Through the examination of temporal patterns, trends, and interrelations, profound insights can be gleaned. However, the presence of anomalies within time-series data [8], [9] poses

a significant challenge, potentially distorting analytical outcomes and leading to erroneous interpretations. In contrast, erroneous data encompasses inaccuracies, noise, or artifacts within the dataset that do not necessarily represent genuine anomalies but instead arise from measurement errors, data corruption, or other sources of noise. While erroneous data can also distort analyses, they are not indicative of genuine deviations from the expected behavior of the system being studied. The analysis of erroneous data has been scoped out in this study.

Anomalies represent points within a time-series dataset that significantly diverge from the anticipated patterns, potentially resulting from diverse influences such as measurement inaccuracies, data corruption, system malfunctions,

The associate editor coordinating the review of this manuscript and approving it for publication was Asadullah Shaikh¹.

TABLE 1. Comparison between 1D CNN-based Siamese and transformer-based siamese networks.

Aspect	1D CNN-Based Siamese	Transformer-Based Siamese
Computational Efficiency	Linear complexity with convolution operations; faster processing, suitable for real-time applications.	Quadratic complexity due to self-attention mechanisms; computationally intensive for long sequences.
Memory Requirements	Lower memory overhead due to fixed-size filters; suitable for low-resource environments.	Higher memory usage due to attention matrices, especially for long sequences.
Temporal Pattern Capture	Effectively captures local temporal patterns and hierarchical features.	Excels in capturing long-range dependencies but may struggle with local patterns without fine-tuning.
Training Simplicity	Fewer hyperparameters; simpler to train and faster convergence.	Requires pre-training or large datasets for generalization; slower convergence.
Interpretability	Better interpretability with localized convolution filters.	Relies on attention heatmaps, making interpretability more abstract.
Suitability for Sequence Length	Ideal for short- to medium-length time-series data.	Better suited for long sequences requiring global context.
Application-Specific Benefits	Robust for anomaly detection in sensor data, ECG signals, and IoT monitoring.	More appropriate for tasks requiring contextual relationships, such as NLP.

or extraordinary occurrences. These anomalies manifest in various forms, including spikes, unexpected trends, outliers, and others [10], [11], [12]. The classification of anomalies is multifaceted, encompassing distinct categories such as point anomalies, contextual anomalies, collective anomalies, and temporal anomalies, each characterized by unique attributes. The detection of anomalies in time-series data assumes critical importance, not solely to uphold data integrity, but also because such anomalous behaviors may signify noteworthy events or phenomena warranting further investigation. Despite the paramount significance of anomaly detection, conventional methodologies encounter inherent limitations stemming from their reliance on labeled data. The scarcity of annotated data points, compounded by the necessity for domain expertise, poses substantial challenges in the acquisition of sufficient labeled data for training [13]. Even when an ample supply of labeled data exists, these methods confront difficulties in identifying contemporaneous or unseen anomalies, aggravated by the perpetual dilemma surrounding the appropriate threshold selection. Moreover, anomalies frequently evolve over time, as exemplified by intruders adapting their tactics in network security contexts, thereby exacerbating the constraints imposed on supervised methodologies, which necessitate continual retraining with updated data to detect emerging anomalies [14]. The acknowledgment of these inherent deficiencies underscores the imperative for innovative and unsupervised approaches to anomaly detection.

Anomaly correction and anomaly smoothing are two distinct strategies used in time-series analysis to address anomalies. Anomaly correction focuses on rectifying detected anomalies to restore data integrity, often necessary for genuine errors or irregularities such as sensor malfunctions or data entry errors. Techniques like data imputation or advanced machine learning models are employed to predict and replace anomalous data points. Conversely, anomaly smoothing aims to mitigate the impact of anomalies on data analysis and visualization without altering the underlying trends. It is typically applied to genuine anomalies representing significant events or natural fluctuations in the system,

using statistical methods like moving averages or outlier detection algorithms to smooth the data while preserving its overall shape. The choice between these strategies depends on the nature of the anomalies and the specific objectives of the analysis. Anomaly detection in time-series and its smoothing have been scoped in this study.

Existing research on anomaly detection in time-series data often faces several limitations, especially when using traditional supervised methods. These methods require large amounts of labeled data, which can be rare and expensive to obtain, limiting their applicability in settings where data labeling is impractical. In addition, they find it difficult to adapt to the evolving nature of anomalies, which may change over time. Traditional models often perform poorly in handling complex data patterns and relationships, and miss complex anomalies that more advanced models such as inspired by Siamese network [15] can detect. In addition, traditional methods are often not robust enough to noisy data, reducing their effectiveness in real-world applications where data errors are common. In addition, Siamese networks are well suited for processing complex data, capturing complex temporal patterns and relationships in data that simpler models may miss. Its robustness to noisy data improves performance in real-world scenarios, making Siamese networks an excellent choice for applications that require high-precision anomaly detection and have limited labeled data.

In this paper, we explore the potential of a Siamese network model based on 1D CNN for anomaly detection in time-series data. Siamese networks, designed for tasks requiring similarity or distance measurements, have demonstrated efficacy across various fields and media types, ranging from images to text, time-series, audio, and video. By leveraging their capacity to learn patterns and recognize deviations, Siamese networks could potentially be utilized for anomaly detection. With the ability to handle both anticipated and unexpected abnormalities, Siamese networks offer a promising approach to addressing the limitations of traditional methods. Recent work, such as TransSiamese [30] has introduced transformer-based Siamese networks for

anomaly detection. However, our method ADSiamNet uses 1D CNN, which is more computationally efficient and more suitable for real-time processing in resource-constrained environments (see Table 1). Moreover, this study has focused on supervised anomaly detection on univariate time-series and the transformer-based models are unsupervised by their nature [67]. Compared with LSTM and GRU, 1D CNN is computationally efficient and requires fewer resources, making it ideal for real-time applications and edge devices [66], [68]. In addition, 1D CNN is well suited for extracting local temporal patterns, which is critical for anomaly detection tasks [20], [21], [22].

This study strictly focuses on supervised anomaly detection in univariate time-series data, and we will extend this work on multivariate data in our future study. Regarding the use of supervised approach for anomaly detection, it is important to recognize that while unsupervised methods are gaining traction, supervised methods continue to be relevant, especially in contexts where labeled data is available and accurate. This research leverages the precision and reliability of supervised learning, arguing its applicability and effectiveness in scenarios where data quality and labeling accuracy are crucial. The choice of a supervised approach, therefore, is aligned with the data characteristics and the specific objectives of this research. The main contributions of our research are as follows:

- The concept of a Siamese network exists (a class of neural networks [16], [17]); however, we use the concept to propose a 1D CNN-based Siamese network (ADSiamNet) model for anomaly detection in time-series as a binary class classification problem. We identify anomalies in time-series, when certain data points are outliers.
- We introduce a smoothing approach to reduce the impact of anomalies on the overall time-series data while preserving the underlying trends and patterns.
- We undertake a comparative analysis of forecasting performance, contrasting two datasets: one containing anomalies (D1) and the other with its smoothed version (D2). We employ various univariate time-series forecasting models in this study to substantiate the hypothesis that D2 exhibits superior forecasting performance compared to D1.
- We validate our anomaly detection algorithm using the physical activity time-series step datasets and NAB benchmark datasets [18], and we will make the experimental codebase public.

The remainder of the paper is structured as follows. In Section II, we review the existing methods in the field. Section III details our contributions and proposed algorithm for anomaly detection and smoothing in time-series. Section IV elaborates on the methodology employed in this study. Section V discusses the experimental setup, results of anomaly detection, anomaly correction, and forecasting outcomes, as well as the significance of our study's results.

Finally, the study concludes in Section VI with an overall summary and potential directions for future research.

II. RELATED WORK

To ensure precision in defining the scope and objectives of our study, it is essential to establish explicit delineations concerning the selection of time-series data and the corresponding methodologies employed for anomaly detection. Our research is singularly focused on investigating anomaly detection within the domain of univariate time-series data. Therefore, the inclusion of multivariate time-series data and associated anomaly detection methods (e.g., space-embedding strategy [19], discord search with local recurrence rates [23]) falls outside the scope of our study. By strictly confining our analysis to univariate time-series data, we ensure a comprehensive and focused exploration of anomaly detection techniques tailored to this specific study.

Anomaly detection methods in univariate time-series can be statistical (e.g., Z-score, percentile-based methods, and Gaussian distribution modeling) [24], threshold-based [9], supervised [25], unsupervised [25], and semi-supervised [25]. The choice of the most suitable anomaly detection method relies on several factors, including the presence of labeled data, the characteristics of the anomalous behavior, the preference for interpretable outcomes, and the trade-off between false positives and false negatives. When selecting an anomaly detection approach, it is essential to carefully consider the specific needs of the application and the resources at hand. Furthermore, each type of detection method comes with its own set of advantages and disadvantages. The decision regarding the most appropriate approach is guided by the availability of labeled data, the nature of the anomalous behavior, the preference for interpretability, and the careful management of false positive and false negative rates. Moreover, state-of-the-art models, such as Transformers, Generative Adversarial Networks (GANs) for anomaly detection, and the latest techniques in transfer learning are relied on the unsupervised technique; therefore, they are excluded for comparative analysis.

Anomaly smoothing aims to mitigate the impact of anomalies on time-series data while preserving its underlying trends and patterns [26], [27]. Anomaly smoothing is typically employed when anomalies are genuine but do not necessarily require correction because they represent significant events, rare occurrences, or natural fluctuations in the system being studied. In such cases, smoothing helps to mitigate the effects of anomalies on data analysis and visualization without altering the fundamental characteristics of the time-series. Anomaly smoothing techniques often involve applying statistical methods, such as moving averages, exponential smoothing, or robust estimators, to smooth out the fluctuations caused by anomalies while retaining the overall shape of the time-series. Additionally, outlier detection algorithms can be used to identify and exclude extreme anomalies that may distort the smoothing process.

In this context, we introduce the ADSiamNet model, designed for the purpose of anomaly detection in univariate time-series. Additionally, we present an approach to address detected anomalies through a smoothing technique based on the quantile method. The incorporation of Siamese networks represents the innovative core of this study. Siamese networks, known for their efficacy in learning from paired examples and distinguishing subtle variances in data, are uniquely suited for anomaly detection in time-series data. This study's application of Siamese networks, coupled with 1D CNNs, addresses a gap in current research, presenting a novel methodological approach. While the supervised nature of the study might appear incremental, the specific implementation of Siamese networks in this context is an innovative step that enriches the existing body of knowledge in anomaly detection. To demonstrate the effectiveness of our approach for anomaly detection, we utilize two time-series activity step datasets, detailed in the Method Section. Furthermore, to validate the applicability of the ADSiamNet model for anomaly detection across various domains, we apply our proposed model to benchmark datasets from the NAB dataset repository.

Supervised anomaly detection methods in univariate time-series rely on labeled data, where anomalies are explicitly identified or known [9], [28]. In the context of these methods, labeled data means that each data point in the dataset must be associated with a label indicating whether it is normal or anomalous. These labels typically follow a binary scheme:

- Normal data, often referred to as “inliers”, is assigned a label of “1”. It represents data points that belong to the normal class, reflecting typical and non-anomalous observations.
- Anomalous data, referred to as “outliers”, is assigned a label of “0”. This category encompasses data points that are rare, unusual, or outliers within the dataset.

During the training phase, supervised anomaly detection methods leverage this labeled data to learn patterns and characteristics specific to normal data. The overarching goal is to construct a model capable of distinguishing between normal and anomalous instances based on these labels. The choice of labels is contingent upon the particular anomaly detection problem at hand. These labels play a pivotal role in both the training and evaluation of the selected anomaly detection model. In this context, machine learning models are developed using labeled anomalous data, with the anomalous behavior either represented as a distinct class or designated by specific anomalous labels. These approaches aim to capture the common patterns of behavior observed within the labeled data and subsequently identify instances that exhibit significant deviations from these established patterns, marking them as exceptions or anomalies. The supervised anomaly detection methods support the following intelligent algorithms [28], [29], [31], [66], [68] – Support Vector Machines (SVM), Random Forests (RF), k-Nearest

Neighbors (k-NN), Decision Trees (DT), Gaussian Mixture Models (GMM), and Deep learning algorithms, such as Artificial Neural Networks (ANN), Multilayer Perceptrons (MLP), Recurrent Neural Networks (RNNs), Long short-term memory (LSTM), Gated recurrent units (GRUs), and CNNs. These models can acquire complex patterns and representational abilities from labeled data that differentiate abnormal instances from normal instances. The primary benefit of supervised machine learning-based methods is that they have a high degree of accuracy when trained on data that is well-labeled. However, it's crucial to recognize that obtaining labeled abnormalities for training is difficult, as abnormalities are typically uncommon and hard to reproduce. As a result, the availability of sufficient labeled data with anomalous values is pivotal in the choice of supervised methods for detecting anomalous behavior. It's crucial to recognize that the detection of anomalous behavior in time-series is a difficult task and necessitates careful consideration of the data properties, domain knowledge, expertise, and the specific goals of the analysis.

Moreover, the unsupervised algorithms [32], [33], [66], [68] (e.g., Elliptic Envelope (EE), Isolation Forest (IF), OC-SVM, Autoencoders, LSTM, GRU, Density-based methods, cluster-based methods, Robust Covariance Estimation, and Deep Generative models) can identify anomalous behavior in univariate time-series without having to rely on explicit definitions. The objective is to learn the typical patterns or structures of normal data and identify instances that deviate significantly from these structures as abnormalities. Unsupervised algorithms can take advantage of both labeled and unlabeled data to identify anomalies; these methods are called semi-supervised [34]. These approaches utilize limited data that is labeled in order to direct the process of anomalous behavior in an unsupervised environment. Such unsupervised methods have difficulties in interpreting and explaining detected anomalies as there are no explicit labels to reference.

Statistical supervised anomaly detection methods in univariate time-series also necessitate labeled data during training [9], [24]. These methods hinge on the underlying distribution of normal data, thus necessitating a balanced representation of both normal and anomalous instances. They are typically characterized by simpler model complexities, relying on statistical metrics and assumptions, which makes them computationally efficient. However, their performance can be heavily reliant on the assumption that normal data follows a particular distribution, such as Gaussian. Departures from this assumption can result in suboptimal outcomes. Generalization capabilities are contingent on the quality and representativeness of the training data, with better performance observed when the statistical assumptions align with the data. These methods are often well-suited for real-time or batch processing, depending on the specific statistical technique employed. Moreover, they can be tailored to specific application domains by selecting appropriate statistical models and relevant features.

1D CNNs, by design, are good at identifying local and short-term patterns within time-series data, owing to their ability to apply convolutional filters effectively over temporal sequences. This characteristic makes them particularly adept at detecting anomalies that manifest as sudden, local deviations from typical patterns. In many practical applications of anomaly detection, such as identifying irregular spikes in financial markets or unusual patterns in sensor data, the focus is predominantly on these short-term irregularities, where 1D CNNs perform robustly. Therefore, the 1D CNN-based Siamese network can be a powerful choice for anomaly detection in time-series data due to its feature learning capabilities, adaptability to a wide range of data distribution, few-shot learning abilities, robustness to noise, model generalization, and scalability, making it well-suited for various real-world anomaly behavior analysis applications in time-series data. Furthermore, 1D CNNs excel at capturing intricate, non-linear data patterns, which proves highly beneficial when anomalies exhibit complex and intricate patterns that linear statistical models struggle to capture. This study strictly focuses on anomaly detection in the labeled data (i.e., supervised approach). A qualitative comparison has been made in Table 2, to demonstrate the key advantages of using ADSiamNet over traditional machine learning and deep learning-based supervised anomaly detection techniques when the data is already labeled. The identified key advantages in Table 2 are traditional benefits of using Siamese networks, and we have incorporated the same in our proposed ADSiamNet model. The advantages of ADSiamNet over other methods depend on the specific anomaly detection task, the characteristics of the data, and the availability of labeled data. Statistical methods can still be effective in cases where data distribution assumptions hold, and when the interpretability of statistical measures has been critical. The choice between well-established models and the 1D-CNN-based Siamese model depends on the specific requirements of the application, including the complexity of the data, the availability of computational resources, and the importance of interpretability in the decision-making process.

The innovation of ADSiamNet resides in its unique application of Siamese networks combined with 1D CNNs specifically for time-series anomaly detection, an area not extensively explored in previous applications of Siamese networks. The model's use of 1D CNNs enables it to effectively capture both local and long-term dependencies within time-series data, addressing a gap where traditional methods may fall short. Its architecture allows for learning from a very limited number of labeled examples through few-shot learning, which is particularly beneficial in anomaly detection where new and evolving anomalies may not have many examples. Additionally, the scalability and efficiency of the architecture make ADSiamNet a notable advancement in the field, offering solutions to some of the persistent challenges in anomaly detection.

The choice of a 1D CNN model in ADSiamNet, while seemingly a straightforward substitution of Conv1D for

Conv2D or Conv3D, is fundamentally driven by the nature of the data being analyzed. Time-series data, characterized by its sequential and one-dimensional structure, calls for a specialized approach that 1D CNNs are adept at handling. The efficacy of 1D CNNs in capturing temporal dependencies and patterns in time-series data is well-established [20], [21], [22], making them a natural choice for this study. The assertion that this choice is not even incremental underestimates the importance of selecting the right tool for the specific characteristics of the data. Furthermore, this research specifically employs 1D CNNs due to their efficiency in processing sequential data and their ability to model temporal correlations with fewer parameters compared to their 2D or 3D counterparts. This efficiency is crucial in anomaly detection tasks, where the speed of detection can be as critical as the accuracy. Thus, the application of 1D CNNs in this context is not just a matter of substituting one type of convolutional layer for another, but a deliberate methodological decision aligned with the nature of the data and the objectives of the anomaly detection task.

III. PROPOSED WORK

This section proposes our solution for anomaly detection and anomaly smoothing from a time-series, with a Siamese network model [35] using the contrastive loss function to enhance the forecasting ability. Our proposed ADSiamNet model is based on the standard architecture of the 1D CNN and has used the concept of a Siamese network (or twin network) for anomaly detection.

A. 1D CNN MODEL

A 1D CNN [36] is a variant of the traditional CNN that is specifically designed to process one-dimensional data, such as time-series, sequences, or signals. While a typical CNN is employed for images, a 1D CNN is appropriate for applications that involve successive data. The network employs convolutional operations across the sequence in order and a set of learnable filters (also known as kernels) to the input data, and produces feature maps that capture different patterns or features present in the input to recognize local patterns and dependencies, followed by incorporation of additional layers, such as activation functions, pooling, and fully connected layers that serve as a means of extracting features and making predictions. Mathematically, the operation performed by a 1D convolutional layer can be represented as follows:

Let X be the input sequence of length L , and let F be the set of filters with size K . Each filter $f_i \in F$ has a set of learnable weights w_i , a bias term b_i , and a stride S_i . The output feature map Y for a single filter f_i is computed as follows, using the 1D convolution operation:

$$Y_i[n] = \sum_{k=0}^{K-1} X[n \cdot S_i + k] \cdot w_i[k] + b_i \quad (1)$$

where: $Y_i[n]$ is the value of the feature map at position n , $X[n \cdot S_i + k]$ represents the value of the input sequence at the

TABLE 2. A qualitative comparison between our proposed ADSiamNet-based and the traditional machine learning and deep learning-based supervised anomaly detection techniques in univariate time-series.

Key Advantages	SVM	RF	k-NN	OC-SVM	DT	MLP	Isolation Forest	Elliptic Envelope	CNN	LSTM	GRU	ADSiamNet
Pairwise Comparison	×	×	×	×	×	×	×	×	×	×	×	✓
Automatically Extracts Discriminative Features	×	×	×	×	×	×	×	×	✓	✓	✓	✓
Flexibility in Handling Anomalies	×	×	×	×	×	×	×	×	✓	✓	✓	✓
Fewer Labeled Anomalies Required	×	×	×	×	×	×	×	×	×	×	×	✓
Adaptability to Changing Data	×	×	×	×	×	×	×	×	✓	✓	✓	✓
Handling Complex Data	×	×	×	×	×	×	×	×	✓	✓	✓	✓
Relationships and Patterns Through Similarity Score	×	×	×	×	×	×	×	×	×	×	×	✓
Feature Learning and Few-shot Learning	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Interpretable Embeddings	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Robustness to Noisy Data	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Capturing Both Local and Long-term Dependencies	×	×	×	×	×	×	×	×	✓	✓	✓	✓
Capturing Complex Temporal Patterns	×	×	×	✓	×	×	×	×	✓	✓	✓	✓
Generalization	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

position $n \cdot S_i + k$, $w_i[k]$ is the weight of the filter f_i at position k , b_i is the bias term of the filter f_i , and S_i is the stride of the filter f_i , which determines how the filter slides along the input sequence.

The feature maps for all filters in F are computed in parallel, resulting in a set of feature maps. These feature maps capture different patterns or features present in the input sequence, and they are then used as input to subsequent layers in the CNN. Equation 1 represents the computation performed by a 1D convolution layer in a CNN. The filter or kernel is convolved with the input sequence, and at each position, the weighted sum of the local receptive field is computed, followed by the application of an activation function to introduce non-linearity in the output. We can modify and expand upon this representation based on specific requirements or include additional equations to represent multiple layers, pooling operations, or other components of a 1D CNN. The benefits of using a 1D CNN for the analysis of time-series data include its capacity to recognize local patterns, have translation invariance, have a hierarchical representation that is learned, share parameters, have noise resistance, and have the potential to be interpretable, all of which are powerful tools for the analysis and extraction of information from time-series data.

Indeed, while 1D CNNs excel in identifying local and short-term patterns within time-series data due to their effective application of convolution filters over temporal sequences, they may overlook long-term dependencies, which could pose challenges for anomaly detection, especially in contexts where anomalies are defined by their relation to extended temporal trends or patterns. This limitation also extends to predictive modeling, where accurate forecasting relies on understanding prolonged temporal relationships.

However, adaptations can enhance 1D CNNs' efficacy in handling long-term dependencies. For instance, integrating dilated convolutions expands the model's receptive field, enabling it to capture longer-term dependencies.

Moreover, stacking multiple convolutional layers facilitates the extraction of complex features, indirectly capturing longer-term trends. Hybrid models combining 1D CNNs with architectures proficient in managing long-term dependencies, such as Recurrent Neural Networks or Long Short-Term Memory networks, could be employed. Furthermore, integrating Siamese network models with 1D CNNs provides a promising avenue for mitigating the limitations of traditional convolutional architectures in capturing long-term dependencies for anomaly detection. Siamese networks operate by comparing pairs of input sequences and learning to distinguish between similar and dissimilar instances. This approach allows the model to capture nuanced similarities and differences across sequences, including long-term dependencies that may not be readily apparent from individual data points. By training the Siamese network on pairs of time-series data, the model can learn to recognize and incorporate long-term temporal relationships into the anomaly detection process. Such integrative approaches can harness the strengths of both architectures and offer a more comprehensive and effective solution, ensuring robust performance across a spectrum of anomaly detection tasks in time-series.

B. PROBLEM FORMULATION WITH CONSTRAINT PROGRAMMING

We formulate the task of anomaly detection in time-series data as a constrained optimization problem. The goal is to identify anomalies while minimizing false positives and false

negatives, ensuring that the model generalizes well across a variety of data patterns and distributions.

1) OBJECTIVE FUNCTION

Let $\mathbf{X} = \{x_1, x_2, \dots, x_T\}$ represent a univariate time-series with T observations, and let $\mathbf{Y} = \{y_1, y_2, \dots, y_T\}$ denote the corresponding binary labels, where:

$$y_t = \begin{cases} 1, & \text{if } x_t \text{ is an anomaly,} \\ 0, & \text{otherwise.} \end{cases}$$

The goal is to learn a decision function $f(\mathbf{X}; \theta)$ parameterized by θ , which minimizes a loss function defined as:

$$\mathcal{L}(\theta) = \sum_{t=1}^T [\ell(y_t, f(x_t; \theta))] + \lambda \Omega(\theta), \quad (2)$$

where: - $\ell(\cdot, \cdot)$ is a binary cross-entropy loss function for classification. - $\Omega(\theta)$ is a regularization term to control model complexity and prevent overfitting. - $\lambda > 0$ is a hyperparameter that balances the loss and regularization.

2) CONSTRAINTS

We impose the following constraints to enforce domain-specific properties:

$$|x_{t+1} - x_t| \leq \delta, \quad \forall t \in \{1, 2, \dots, T-1\}, \quad (3)$$

$$y_t \in \{0, 1\}, \quad \forall t, \quad (4)$$

where δ is a threshold for temporal smoothness to avoid mis-classifications due to noise.

The proposed problem thus seeks a solution that satisfies the constraints while optimizing the objective function, balancing accuracy and computational feasibility. This approach is particularly useful for incorporating physical constraints such as allowed range and temporal consistency into the learning process.

3) PROBLEM CLASSIFICATION

The constrained anomaly detection problem can be viewed as a binary classification task with dependencies, which is NP-hard [65] in nature. Evaluating all possible subsets of anomaly points in time series data involves a combinatorial search process, which results in exponential computational complexity. Furthermore, considering constraints such as temporal smoothness introduces additional dependencies that enhance the optimization challenge. To address this complexity, the proposed approach leverages a Siamese neural network architecture. By transforming the anomaly detection task into a pairwise similarity evaluation problem, the method effectively reduces the search space. By leveraging learned feature representations, it not only simplifies the optimization process but also increases computational efficiency by enabling faster convergence during training.

C. ADSiamNet MODEL

The 1D convolutional twin neural network structure of ADSiamNet provides a robust framework for anomaly

detection by leveraging local feature extraction, weight sharing, dimensionality reduction, and effective similarity learning. Further theoretical elaboration on these points strengthens the rationale for choosing this architecture in predictive analysis of time-series data.

Lets consider two time-series sequences $X_1 = [x_{1,1}, x_{1,2}, \dots, x_{1,n}]$ and $X_2 = [x_{2,1}, x_{2,2}, \dots, x_{2,n}]$, where each sequence represents a set of observations over time. The goal of the ADSiamNet model is to determine whether these sequences are similar or dissimilar, which aids in detecting anomalies.

We define the hypothesis H_0 and H_1 as follows:

$H_0 : X_1$ and X_2 are similar (normal)

$H_1 : X_1$ and X_2 are dissimilar (anomalous)

Let $\mathcal{X}$ be the set of all possible time-series sequences. We aim to learn a mapping $f : \mathcal{X} \rightarrow \mathcal{H}$, where $\mathcal{H}$ is a lower-dimensional embedding space. The shared convolutional neural network (CNN) f learns this mapping such that:

$$\forall X_1, X_2 \in \mathcal{X}, \text{ if } X_1 \text{ and } X_2 \text{ are similar,} \\ \text{then } \|f(X_1) - f(X_2)\| \text{ is small} \quad (5)$$

$$\text{if } X_1 \text{ and } X_2 \text{ are dissimilar,} \\ \text{then } \|f(X_1) - f(X_2)\| \text{ is large} \quad (6)$$

Let $\mathcal{X}$ be the set of all possible time-series sequences. We aim to learn a mapping $f : \mathcal{X} \rightarrow \mathcal{H}$, where $\mathcal{H}$ is a lower-dimensional embedding space. The shared convolutional neural network (CNN) f learns this mapping such that:

$$\forall X_1, X_2 \in \mathcal{X}, \text{ if } X_1 \text{ and } X_2 \\ \text{are similar, then } \|f(X_1) - f(X_2)\| \text{ is small} \\ \text{if } X_1 \text{ and } X_2 \text{ are dissimilar, then } \|f(X_1) - f(X_2)\| \text{ is large} \quad (7)$$

The **Algorithm 1** represents the steps in the ADSiamNet model for predictive analysis in time-series data. This **Algorithm 1** represents the computation performed by ADSiamNet, where two input sequences (time-series) are passed through parallel branches of the network, sharing the same weights. The convolutional layers, activation functions, pooling, and flattening operations are applied to each branch independently. In the end, the input time-series are classified based on the distance metric and identified features for certain predictive analyses. The network is trained to learn embeddings to input pairs in such a way that similar inputs are closer in the embedding space, while dissimilar inputs are farther apart.

The input layer takes in two 1D sequences, denoted as X_1 and X_2 , which are the inputs to the Siamese network. The convolutional layers consist of filters that slide across the input sequences to extract local features. The convolutional operation is performed by taking the dot product between the filters and overlapping segments of the input sequences.

Algorithm 1 ADSiamNet Model for Predictive Analysis in Time-Series**Require:**

First time-series sequence: $X_1 = [x_{1,1}, x_{1,2}, \dots, x_{1,n}]$
 Second time-series sequence: $X_2 = [x_{2,1}, x_{2,2}, \dots, x_{2,n}]$
 f : Shared Convolutional Neural Network (CNN)
 h_1 and h_2 : Feature vectors obtained by passing x_1 and x_2 through the shared CNN, respectively
 S : Similarity function that measures the similarity between h_1 and h_2
 L : Loss function used for training, typically contrastive loss
 P : Pairs of samples (x_1, x_2) with corresponding labels y , where $y = 1$ indicates similar samples, and $y = 0$ indicates dissimilar samples.

Ensure:

- Prediction in time-series with learned embeddings to classify similar and dissimilar samples: C
- 1: Apply convolutional filters to X_1 and X_2 with a label y to obtain h_1 and h_2
 - 2: Apply activation function to the convolutional outputs
 - 3: Apply pooling operation to obtain feature maps
 - 4: Flatten the feature maps
 - 5: Calculate the similarity score $S(x_1, x_2) = \text{Similarity}(h_1, h_2)$
 - 6: Compute the loss $L(x_1, x_2, y)$ using the contrastive loss function
 - 7: Update the weights and biases of the network to minimize $L(x_1, x_2, y)$
 - 8: Make the prediction (C) based on the loss value and identified features

The output of the convolutional layers is a set of feature maps. The activation function (e.g., ReLU, SeLU, tanh) is applied element-wise to the feature maps to introduce non-linearity and capture complex patterns and relationships in the data. The output of the activation function is a set of activated feature maps. The pooling layers (e.g., max pooling or average pooling) reduce the dimension of the activated feature maps by downsampling. The output of the pooling layers is a set of pooled feature maps. The flattening step converts the 2D pooled feature maps into a 1D vector and concatenates all the features into a single long vector, allowing for further processing. The flattened feature vectors from the two input sequences are concatenated into a single feature vector for subsequent layers. The fully connected layers consist of weights and biases that transform the concatenated feature vector. The weights and biases are learned through training. The FC layers apply linear transformations followed by an activation function to capture high-level features. The output layer produces the final prediction or output (C) based on the transformed features from the fully connected layers. The output is a binary classification (e.g., anomaly vs. normal) to indicate

the anomaly level. Each layer in the ADSiamNet is crucial in obtaining and altering the input data, this enables the network to learn and make predictions based on the patterns and properties present in the time-series data. The contrastive loss function used in the algorithm encourages similar inputs to have a higher similarity score and dissimilar inputs to have a lower similarity score.

The mathematical formulation of the ADSiamNet model includes several key operations. The convolutional operation is performed as follows:

$$h_{1,i} = \sigma \left(\sum_{k=1}^m w_k \cdot x_{1,i+k-1} + b \right) \quad (8)$$

$$h_{2,i} = \sigma \left(\sum_{k=1}^m w_k \cdot x_{2,i+k-1} + b \right) \quad (9)$$

where w_k are the convolutional filter weights, b is the bias term, and σ is the activation function.

Next, the pooling operation is applied to reduce the dimensionality of the feature maps:

$$p_{1,j} = \max(h_{1,(j-1)s+1}, \dots, h_{1,js}) \quad (10)$$

$$p_{2,j} = \max(h_{2,(j-1)s+1}, \dots, h_{2,js}) \quad (11)$$

where s is the stride of the pooling window.

Following pooling, the feature maps are flattened to form feature vectors:

$$\mathbf{h}_1 = \text{Flatten}(p_1) \quad (12)$$

$$\mathbf{h}_2 = \text{Flatten}(p_2) \quad (13)$$

The similarity between the two sequences is then calculated using a similarity function, commonly the Euclidean distance:

$$S(X_1, X_2) = \text{Similarity}(\mathbf{h}_1, \mathbf{h}_2) \quad (14)$$

$$S(X_1, X_2) = \|\mathbf{h}_1 - \mathbf{h}_2\| \quad (15)$$

Finally, the loss function used to train the network is the contrastive loss, defined as:

$$L(X_1, X_2, y) = y \cdot \frac{1}{2} S(X_1, X_2)^2 + (1 - y) \cdot \frac{1}{2} \max(0, m - S(X_1, X_2))^2 \quad (16)$$

where y is the label indicating similarity (1 if similar, 0 if dissimilar) and m is a margin parameter.

D. PROPOSED ALGORITHM FOR ANOMALY DETECTION AND SMOOTHING

We have proposed **Algorithm 2** for time-series data loading, preparation of sliding windows, data labeling, anomaly detection, and smoothing. The algorithm consists of two parts, — anomaly detection (to correctly identify x_i under H_1) and anomaly smoothing. We have defined the set of sliding windows as:

$$W = \{w_1, w_2, \dots, w_m\} \quad (17)$$

where each sliding window $w_i \subset X$ is of size W . The task is to measure the similarity between these windows using a Siamese neural network.

Let $S : W \times W \rightarrow \mathbb{R}$ be a similarity function such that $S(w_i, w_j)$ computes the similarity score between windows w_i and w_j . In our case, this similarity measure is derived from the Siamese network, which is trained to minimize the contrastive loss function L defined as:

$$L(w_i, w_j) = (1 - y) \cdot \frac{1}{2} D^2 + y \cdot \frac{1}{2} \max(0, m - D)^2 \quad (18)$$

where D is the Euclidean distance between the embeddings of w_i and w_j , y is a binary label indicating whether w_i and w_j are similar ($y = 0$) or dissimilar ($y = 1$), and m is a margin parameter.

The Siamese network maps sliding windows to a latent space where similar windows are closer. Define the embedding function as $E : W \rightarrow \mathbb{R}^d$, where d is the dimension of the embedding space. The distance D between two windows w_i and w_j is:

$$D(w_i, w_j) = \|E(w_i) - E(w_j)\| \quad (19)$$

By setting a threshold T on this distance, anomalies can be identified:

$$L_{AD} = \{(w_i, 1) \mid D(w_i, w_j) > T\} \cup \{(w_i, 0) \mid D(w_i, w_j) \leq T\} \quad (20)$$

The sliding window methodology entails the systematic movement of a fixed-size window across an input time-series dataset sequentially, facilitating localized operations at each position. This computational procedure delineates the sequential steps involved in loading time-series data, preparing sliding windows of various sizes (e.g., 25, 50, 75), labeling data within each sliding window, detecting anomalies utilizing the ADSiamNet model, and subsequently smoothing anomalies via a quantile-based approach. It begins by loading the time-series data and splitting it into sliding windows of a specified size. Then, it initializes an empty list for labeled data. For each sliding window, a reference window is randomly selected, and the distance between the sliding window and the reference window is calculated using the Siamese network. If the distance exceeds a predefined anomaly threshold, the window is labeled as an anomaly; otherwise, it's labeled as normal. Anomaly-smoothed data is then extracted by selecting only the windows labeled as normal. Finally, the algorithm returns the anomaly-smoothed time-series data. The Siamese network maps sliding windows to a latent space where similar windows are closer. For a given quantile $Q \in [0, 1]$, the smoothing function is:

$$S(W) = \frac{1}{|W|} \sum_{x_i \in W} x_i \quad (21)$$

Anomalies are removed, and only normal data points contribute to the smoothed value:

$$Y = \{S(W_i) \mid (W_i, 0) \in L_{AD}\} \quad (22)$$

Algorithm 2 Time-Series Anomaly Detection With ADSiamNet Model and Followed by Smoothing With a Quantile Approach

Require:

Input time-series data: $X = [x_1, x_2, \dots, x_n]$
 Sliding Window size: W
 Anomaly threshold: T

Ensure:

Output anomaly-smoothed time-series data: $Y = [y_1, y_2, \dots, y_n]$

- 1: Load the labeled time-series data X
 - 2: Split X into sliding windows of size W
 - 3: Initialize an empty list for anomaly detection labels in W : $L_{AD} = []$
 - 4:
 - 5: **Part-A: Anomaly detection in W using the Siamese network**
 - 6: **for** each sliding window w in X **do**
 - 7: Select a reference sliding window W randomly from X
 - 8: Calculate the distance D between w and W using the Siamese network
 - 9: **if** $D > T$ **then**
 - 10: Label w as an anomaly: $L_{AD} \leftarrow L_{AD} \cup \{(W, 1)\}$
 - 11: **else**
 - 12: Label w as normal: $L_{AD} \leftarrow L_{AD} \cup \{(W, 0)\}$
 - 13:
 - 14: **Part-B: Anomaly smoothed based on a quantile approach**
 - 15: Quantile: $Q \in \{0, 1\}$
 - 16: Initialize an empty list for anomaly-free (or smoothed) data: $Y = []$
 - 17: **for** each labeled data $(W, label)$ in L_{AD} **do**
 - 18: Remove anomaly by smoothing with average calculation in the selected window size W
 - 19: **if** $label = 0$ **then**
 - 20: Add W to Y (anomaly-smoothed data) after rounding off
 - 21: **return** Y
-

This approach ensures that outliers do not influence the final smoothed time-series data.

The Conv1D layers in the Siamese network allow for the extraction of local features from the time-series data. Given a window w_i , the convolution operation can be expressed as:

$$f_k(w_i) = \sigma \left(\sum_{j=1}^W w_{k,j} \cdot x_{i+j-1} + b_k \right) \quad (23)$$

where f_k is the feature map, $w_{k,j}$ is the kernel weight, b_k is the bias term, and σ is the activation function. This operation ensures that local temporal patterns are effectively captured.

With multiple Conv1D and pooling layers, the network captures increasingly abstract features, crucial for distinguishing between normal and anomalous patterns. Mathemat-

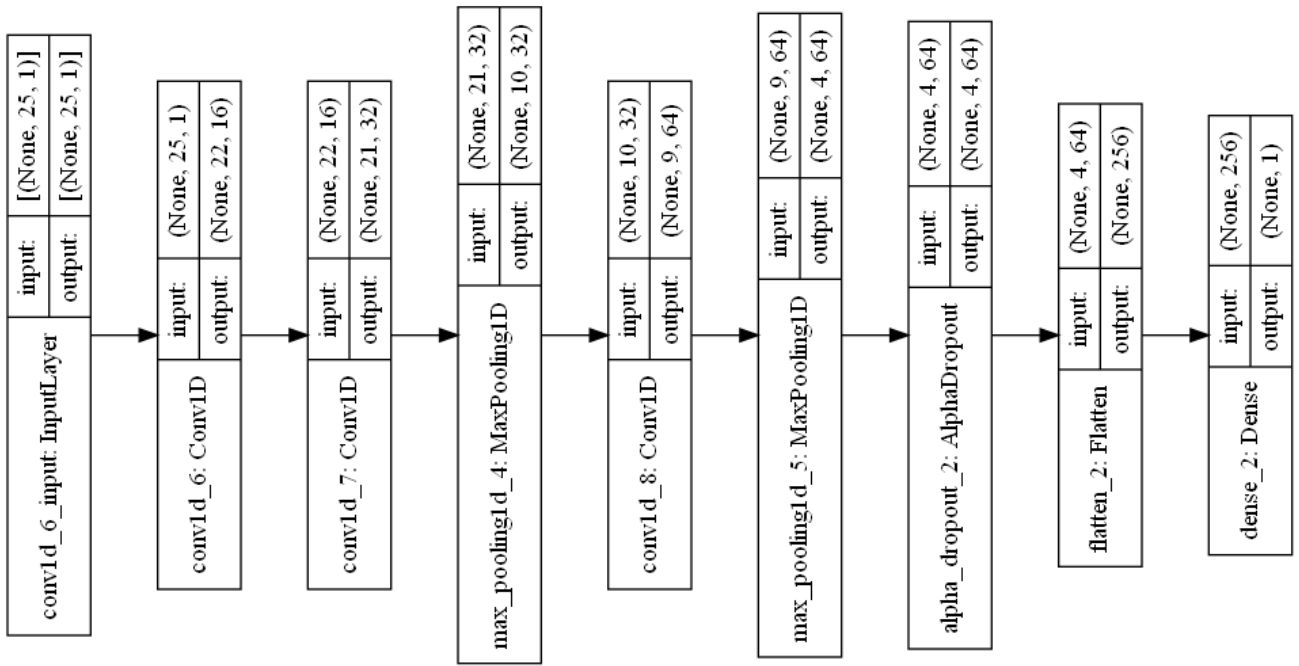


FIGURE 1. The complete model structure of the proposed ADSiamNet model.

ically, let F_l be the feature representation at layer l :

$$F_{l+1} = \text{Pooling}(\sigma(F_l * W_l + B_l)) \quad (24)$$

where $*$ denotes convolution, and W_l and B_l are the weights and biases at layer l . This hierarchical feature learning is beneficial for complex time-series data.

The summary and complete structure of the proposed ADSiamNet model are depicted in Figure 1. The proposed model is sequential and consists of eight layers, with an input 1D convolution (Conv1D) layer to extract local features from the input data. The initial two ($n=16$ and $n=32$) and the fourth ($n=64$) layers are Conv1D layers to capture more abstract features from the data, the third and the fifth layers are the max pooling layer to select the maximum features from the feature map covered by the filter, the sixth layer is the alpha dropout layer with a factor of 0.1 for regularization to prevent overfitting, the seventh layer is the flattening layer to convert a multidimensional array into a single 1D vector, and the final dense layer is responsible for classifying time-series data (anomalous or non-anomalous) based on output from the flattening layer. The softmax activation ensures that the output represents probability scores for each class, and you can choose the class with the highest probability as the prediction. “n” signifies the total number of neurons in the respective layers. We have used the kernel size as 4, 2, and 2 for the first, second, and fourth Conv1D layers, respectively with the SeLU activation function [37], [38]. With SeLU, MaxPooling1D [38], [39], and AlphaDropout [38], [39] are applied to the input sequence.

Furthermore, we have used Adam optimizer in this study to compile the proposed ADSiamNet model. The Adam

optimization algorithm [38] combines the advantages of both RMSprop and momentum methods. Adam has a consistent average of the first and second moments of the parameters’ gradients. It employs these average moves to alter the parameters during training. The algorithm changes the rate of learning for each parameter based on the magnitude of the previous gradient. This rate of learning is adaptive, which allows Adam to have a faster rate of convergence and to effectively deal with sparse vectors of data. The default learning rate (α) of the Adam optimizer in Keras is 0.001 [38]. We have used Early Stopping [40] and Exponential Decay [41] with the Adam optimization function to compile our ADSiamNet model. Adam incorporates both momentum (through the first moment) and adaptive learning rates (through the second moment) to update the model’s parameters. The bias correction terms correct for the initial biases of the moments at the beginning of training.

We have used a quantile approach to smooth the detected anomalies as outliers in the time-series. For this experiment, we have considered a quantile boundary of 0.95 and a sliding window size of 25 to get rid of unnecessary outliers in the specific window. Moreover, a contrastive loss function [42] has been used in the ADSiamNet the model’s ability to compare similar and dissimilar data points.

Furthermore, choosing an appropriate threshold is important because it determines the sensitivity and specificity of the anomaly detection process. Typically, this threshold can be determined based on the statistical properties of the data, such as the mean and standard deviation of the distances in the training samples, or using a percentile value that best distinguishes normal data from anomaly data based

on historical observations. In addition, the threshold can be optimized using cross-validation techniques on a labeled dataset to achieve a balance between false positives and false negatives.

E. TIME COMPLEXITY ANALYSIS OF THE PROPOSED ALGORITHM

The complexity of the algorithm's time is often expressed using Big O notation [43], which is intended to provide a maximum rate of growth for the algorithm as the input size increases. The time complexity of the proposed **Algorithm 2** has been analyzed as follows :

- **Data Loading:** The time complexity of loading the time-series data depends on the specific implementation and the size of the data. Assuming a linear scanning of the data, the time complexity is typically $O(n)$, where n is the number of data points in the time-series.
- **Sliding Window Preparation:** Splitting the time-series data into sliding windows of size w involves iterating over the data once. The time complexity is $O(n)$ since each data point is visited once.
- **Data Labeling:** The data labeling step involves calculating the distance between each sliding window and a reference window using the Siamese network. This process is performed for each sliding window, resulting in $O(n)$ calculations.
- **Anomaly Detection:** The time complexity of anomaly detection depends on the complexity of the Siamese network's forward pass for each sliding window. If the Siamese network has a constant time complexity for a single forward pass, then the time complexity is $O(n)$.
- **Anomaly smoothing:** The time complexity of anomaly smoothing depends on the specific technique employed and the number of anomalies detected. In the proposed algorithm, anomaly smoothing involves quantile approach and iterating over the labeled data and selecting only the normal windows, resulting in $O(n)$ time complexity.

Overall, the dominant factor in the computational cost of the algorithm is the iteration over the entire dataset, resulting in a complexity of $O(n)$, where n is the number of data points in the time-series. Therefore, the computational cost of the provided algorithm is linear in the number of data points in the input time-series data. It's important to note that the time complexity may differ depending on the specifics of the implementation and the complexity of the Siamese network.

F. SINGLE-PASS LEARNING VS. ADSIAMNET MODEL FOR ANOMALY DETECTION

Single-pass learning techniques are crucial when dealing with real-time or streaming data, scenarios where it's impractical to repeatedly process the entire dataset. In a 'single pass of data,' the dataset is processed just once during training. This approach assumes that we have access to the entire dataset at once and can perform multiple passes if needed.

In the context of supervised learning, we leverage labeled data to train a model capable of making predictions on the latest and unseen data. The model acquires patterns and relationships from the training data during this single pass. Conversely, the ADSiamNet model represents a specialized neural network architecture used for tasks like similarity learning and anomaly detection. The network's objective is to learn a similarity metric between pairs of data points. Anomalies are identified by assessing the dissimilarity or distance between data points. If two data points exhibit significant dissimilarity, it may suggest the presence of an anomaly. The choice between a traditional supervised learning approach with a single pass of data and opting for a Siamese network for anomaly detection hinges on the specific problem and dataset characteristics. If we possess labeled data and aim to detect anomalies by leveraging learned similarities or dissimilarities between data points, a Siamese network may be a suitable choice, as is the case in our scenario.

IV. METHODS

This section outlines the specific methods employed to execute the overall experiment, as depicted in Figure 2. These methods encompass data collection, ethical approval for the datasets, data preprocessing, the evaluation of similarity scoring within ADSiamNet, the choice of activation and optimization functions for the ADSiamNet model to predict anomalies in time-series data, the selection of machine learning and deep learning-based supervised anomaly detection methods for performance comparison with ADSiamNet model, the selection of univariate time-series models used for comparative forecasting analysis between anomaly-smoothed data and anomalous data, and the application of performance metrics to assess prediction and forecasting effectiveness. Our research introduces a novel and innovative method for detecting anomalies in time-series data, employing the ADSiamNet model. We underscore the significance of precise anomaly detection, showcase the efficacy of our methodology using real-world data, and furnish proof that smoothing anomalies enhances the performance of time-series forecasting models. These findings hold relevance across various applications where the analysis of time-series data plays a pivotal role.

A. DATA SETS

We have conducted our experiments on the real-time MOX2-5 activity step dataset, publicly available physical activity step dataset [44] and NAB anomaly datasets [18]. The physical activity univariate time-series datasets has been used for anomaly detection and smoothing. The public benchmark NAB dataset has been utilized for anomaly detection and performance evaluation of our proposed ADSiamNet model. We have followed ethical guidelines during the collection of MOX2-5 activity step dataset from adults (aged 18–64). Bodybuilders, pregnant individuals, and those with a

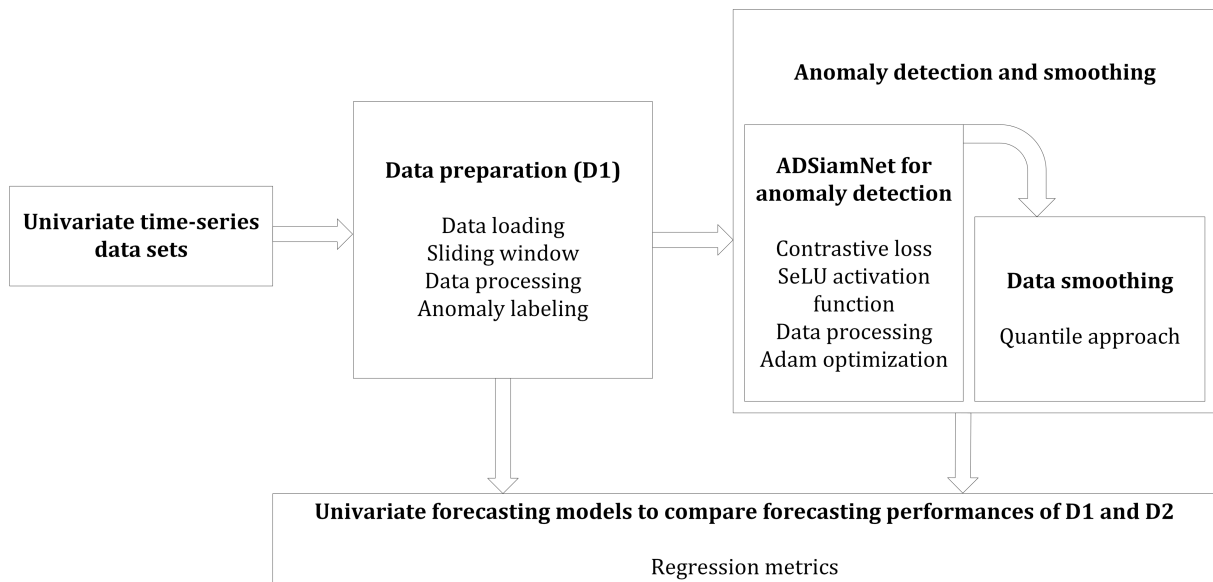


FIGURE 2. The adopted methodology for the overall experiment.

significant medical history or chronic illnesses were not included in the study.

1) REAL-TIME MOX2-5 ACTIVITY DATASET

We anonymously collected physical activity data over months from a cohort of 16 adults (12 male and 4 female) residing in Grimstad, Norway. The data was gathered using the wearable activity sensor MOX2-5, which is CE certified. Adhering to Norwegian ethical guidelines, real-time activity data was collected from consenting participants. The personal data of participants were collected and evaluated for compliance with data protection laws, with approval obtained from the Norwegian Study Data Center (NSD). Participant characteristics with datasets are detailed in [45]. As illustrated in Figure 3, the activity data of participant P1, along with corresponding anomalies, serve as a representative example in this study.

Data on physical activity from MOX2-5 sensors was continuously gathered throughout the day using Bluetooth (BLE) short-range wireless technology. This technology adheres to a fixed sampling rate, typically set at around 1 Hz (1 sample per second), and stores the data in comma-separated values (CSV) format. Sampling and recording of the data occurred at brief intervals, often in real-time or near-real-time.

2) PUBLIC ACTIGRAPHY ACTIVITY DATASET

The physical activity dataset [44] consists of two distinct folders: one containing data pertaining to control subjects and another housing data for the conditioned group. This dataset provides a detailed protocol for the acquisition of physical activity data using ActiGraph devices. The sampling frequency employed was 32Hz, with a maximum displacement of 0.05 g. Activities were recorded as counts in the memory of the ActiGraph watch, with count volume

proportional to the intensity of movement. Continuous activity levels were recorded at one-minute intervals. Each participant's data is provided in a file containing timestamp (divided into one-minute intervals), date of measurement, and activity level recorded by the ActiGraph watch. Additionally, the authors have labeled the data with binary indicators (1 for anomaly, 0 for normal) based on predefined thresholds or outliers in daily step counts. The dataset comprises an average of 55,593 records per participant. While the analysis encompasses all participants' data, this study focuses on results obtained from a single participant's data (referred to as P1) to streamline representation and enhance interpretability. As illustrated in Figure 4, the activity data of P1, along with corresponding anomalies, serve as a representative example in this study.

3) NAB DATASET FOR BENCHMARKING

NAB [46] repository is composed of over 50 real-world and artificial time-series data files from different domains. It contains both anomalous and non-anomalous data. The NAB datasets are intended to specifically evaluate anomalous behavior detection methods. It contains a variety of real-world time-series data that are relevant to the server, such as sensor readings, server metrics, and network traffic, with an anomalous behavior label. The dataset encompasses multiple domains, including environmental monitoring, financial management, and IT infrastructure. Existing performances of the anomaly detectors in the NAB dataset have been captured by [18].

B. ETHICAL APPROVAL

We have used publicly available depression (with physical activity records) [44] and NAB datasets [18]; therefore,

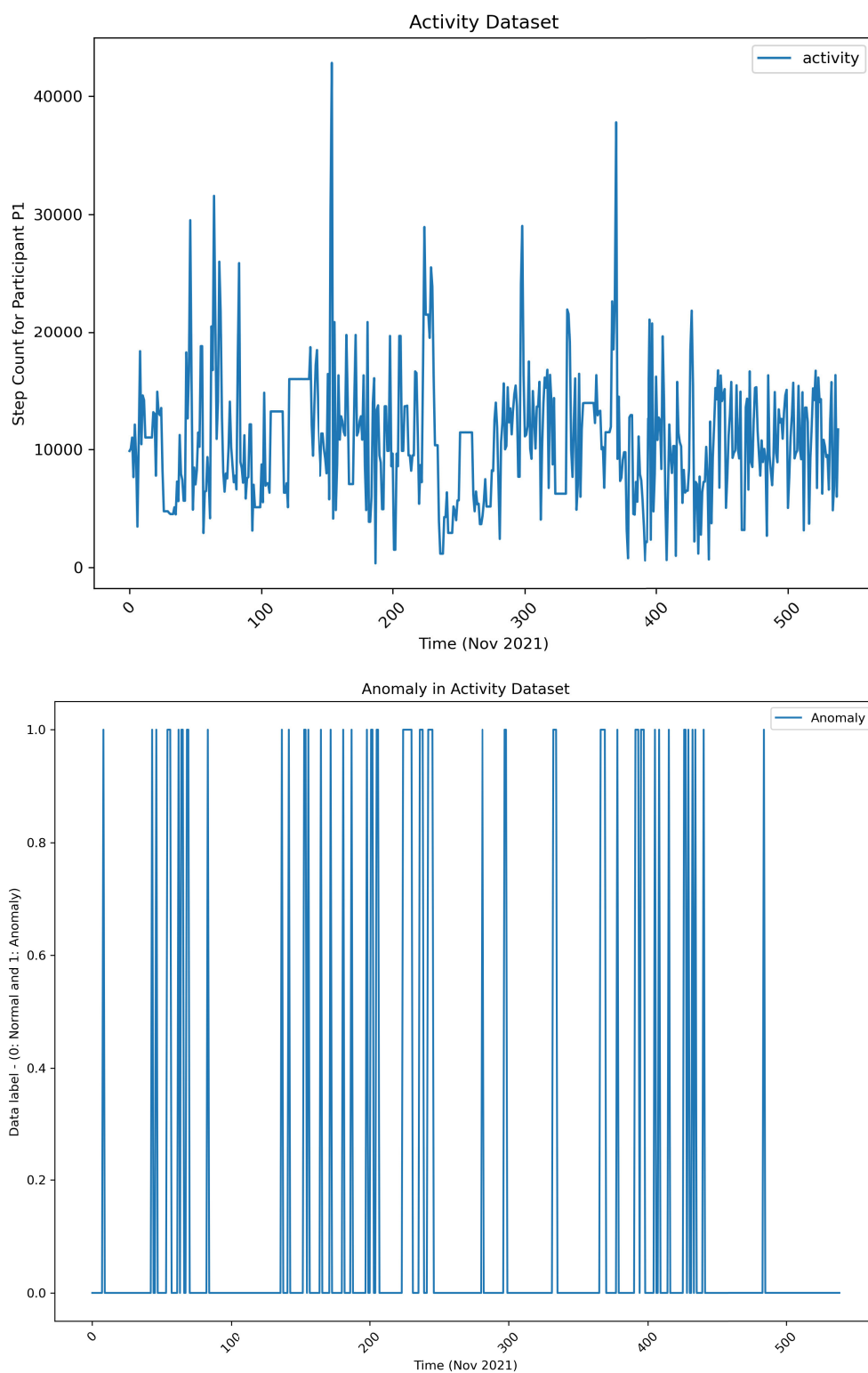


FIGURE 3. (a) The summary of the MOX2-5 activity data for Participant P1. (b) The summary of the anomaly levels in MOX2-5 activity data for Participant P1.

no ethical approval has been required for the same. For MOX2-5 activity data collection and management,

we obtained approval from the Norwegian Centre for Research Data (NSD) (797208) and obtained ethical clear-

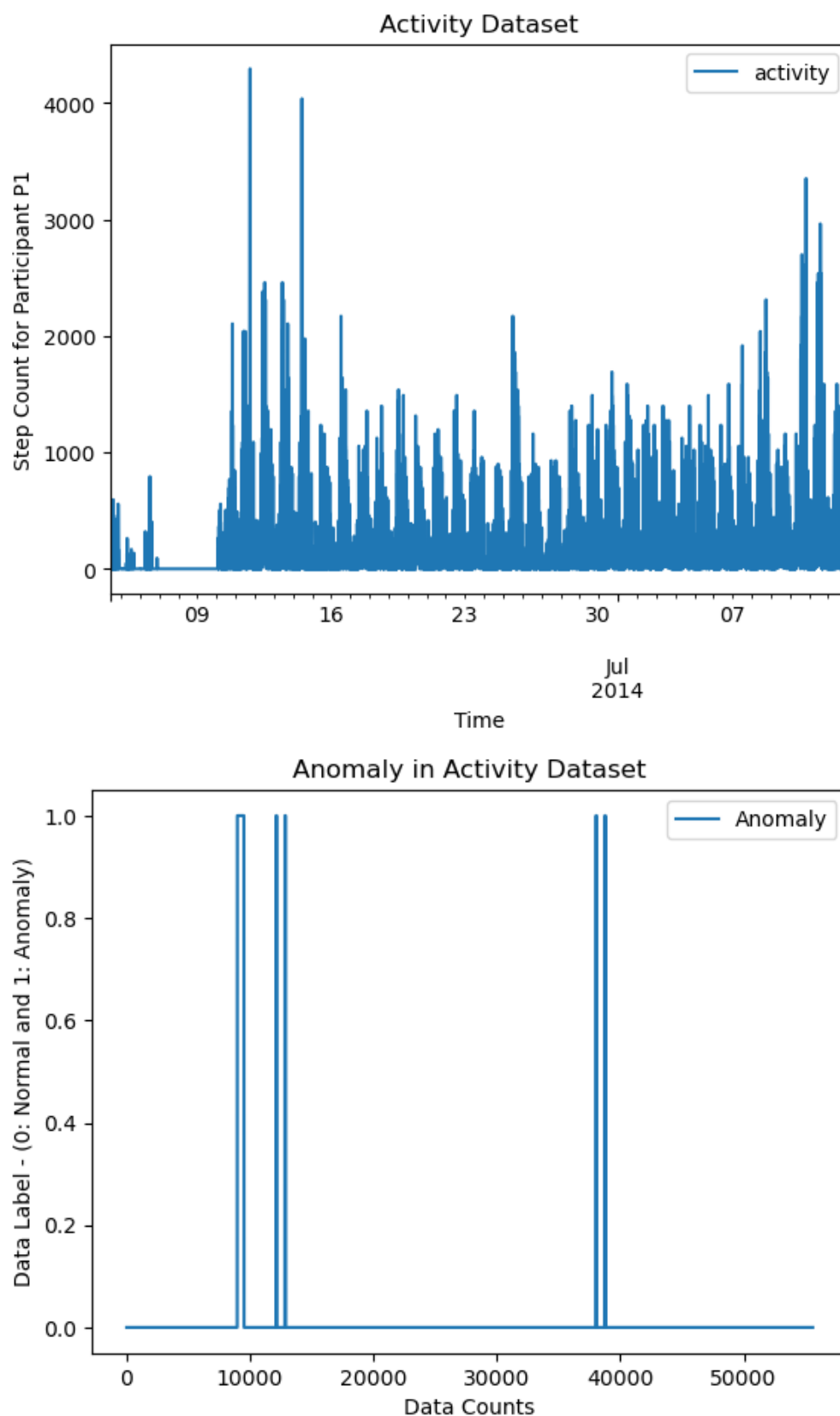


FIGURE 4. (a) The summary of the actigraphy activity data for Participant P1. (b) The summary of the anomaly levels in actigraphy activity data for Participant P1.

ance from the Regional Committees for Medical and Health Research Ethics (REK) (53224). Participation in the study was voluntary, and all participants provided informed or signed consent. Additionally, we ensured the confidentiality of participant data by avoiding the disclosure of any identifiable information, including numbers, text, or figures.

C. DATA PREPARATION

The data preparation steps for data loading, windowing, and anomaly labeling in a window for time-series data are summarized as follows :

- **Data loading:** We loaded the raw time-series data into memory from a data source such as a CSV file. We used reading the data into a pandas Data Frame.
- **Sliding window:** We defined the window size (e.g., the default length of 25) and a stride length of 1. We iterated over the time-series data with the defined stride length, and for each iteration, we extracted a window of the specified size.
- **Data Processing:** We computed the summation of anomaly labels in a window. Furthermore, we corrected missing values removed trend and seasonality from the data, and converted the data to stationary. We used a standardization method to normalize the time-series for forecasting with standard statistical and deep learning models. Standardization, also known as z-score normalization, is a common preprocessing technique used in statistics and machine learning to transform data so that it has zero mean and unit variance.
- **Anomaly labeling:** We determined the ground truth labels for anomalies within each window. Anomalies can be labeled based on pre-existing knowledge, expert judgment, or by using anomaly detection algorithms. We assigned a binary label (e.g., 1 for normal, 0 for anomaly) to each sliding window, indicating the presence or absence of an anomaly within that window. The process of anomaly labeling may vary depending on the nature of the anomalies and the available information.

D. MODELS FOR ANOMALY DETECTION ON LABELED DATA

The machine learning and deep learning model selection hinges on data characteristics and specific anomaly detection task requirements. Well-established anomaly detection models, such as Isolation Forest [57], KNN [58], One-SVM [59], Decision Tree (DT) [25], [47], [62], Random Forest (RF) [60], Elliptic Envelope [61], MLP [63], 1D CNN [64], LSTM [66], [68], and GRU [66], [68] can demonstrate effectiveness when labeled data is accessible, and enable them to learn from both normal and anomalous examples in training. Therefore, we employed these models for a comparative performance analysis against ADSiamNet on the Activity Dataset. There are many more anomaly detection models; however, our selected models for performance comparison are popular, mostly used, and well-known.

The *Isolation Forest* is an ensemble learning algorithm designed for efficient anomaly isolation. It functions by randomly selecting features and dividing data points until anomalies are separated along shorter paths. This method is particularly well-suited for high-dimensional data and adaptable to varying data distributions. The *KNN* algorithm classifies data points based on the majority class among their *k*-nearest neighbors. For anomaly detection, data points that are distant from their neighbors are identified as anomalies. *KNN*'s performance hinges on the choice of the parameter *k*. *One-SVM* is a powerful classification algorithm that can be repurposed for anomaly detection. *SVM* separates data points into two classes by identifying a hyperplane with the maximum margin. Anomalies are often characterized as data points situated on the "incorrect" side of this hyperplane. *DT* are supervised models that can be adapted for anomaly detection tasks by splitting data into branches based on features and creating a tree structure. *DT* is interpretable and can be useful for detecting anomalies when the data has clear decision boundaries. *Random Forest* employs an ensemble of decision trees to uncover anomalies in labeled data by harnessing the collective predictive strength of multiple trees, enhancing result reliability. It evaluates feature significance within the ensemble, enabling the identification of pivotal variables crucial for anomaly detection, thereby facilitating insights into underlying data patterns. Additionally, *Random Forest*'s scalability to large datasets and its applicability across diverse domains render it a versatile solution for anomaly detection in labeled data tasks. The *Elliptic Envelope* method operates under the assumption that the majority of data adheres to a multivariate Gaussian distribution. It proceeds by fitting an ellipse around the central data points and flagging points lying outside the ellipse as anomalies. This approach is valuable for uncovering global outliers within structured data. An *MLP*, a type of multi-layer feed-forward neural network, is employed to capture intricate data patterns. It relies on labeled data containing both normal and anomalous instances during training. Anomalies are identified by comparing model predictions to a predefined threshold, marking instances above this threshold as anomalies. *1D CNN*-based supervised anomaly detection entails training a neural network with labeled data to differentiate between normal and anomalous time-series patterns. This CNN is trained to identify anomalies by learning from labeled instances, making it well-suited for scenarios where annotated training data is accessible and anomalies can be detected through pattern recognition in univariate time-series data. *LSTM* networks are a type of RNN designed to capture long-term dependencies in sequential data. They are well suited for modeling temporal patterns by maintaining memory through gating mechanisms that regulate the flow of information. For anomaly detection in univariate time series, *LSTMs* leverage their ability to retain past information and detect deviations from learned patterns, making them effective in detecting trends and temporal anomalies. *GRUs* are simplified variants of *LSTMs* that aim to achieve similar performance with fewer

parameters. They use gating mechanisms to manage memory and update state, enabling efficient learning of sequential dependencies. GRUs are less computationally intensive than LSTMs, making them well suited for time series anomaly detection tasks, especially when dealing with datasets that require faster training and inference. Both LSTMs and GRUs are good for learning temporal dynamics, identifying anomalies based on deviations from expected sequences, and handling complex time series data.

For the NAB benchmark datasets, we compared the performance of ADSiamNet with the anomaly detectors as mentioned in [18].

E. TIME-SERIES MODELS FOR UNIVARIATE FORECASTING

The choice between univariate time-series forecasting models depends on the specific objectives and characteristics of the time-series data. Determining the most appropriate prediction method for a given data set often requires experimentation and model evaluation. We incorporated the following time-series models for univariate forecasting to do a comparative performance analysis of the models with datasets (D1: with anomaly and D2: anomaly smoothed) against the MSE metrics –

- **LSTM:** Univariate LSTM models are specifically designed to handle time-series data with a single variable or feature. When focusing on a single variable and leveraging its historical patterns, univariate LSTM models offer simplicity and efficiency in time-series analysis. *LSTM* [48] is a type of RNN with specialized memory cells, allowing it to effectively capture long-range dependencies in sequential data. LSTM's flexibility in handling variable-length input and output sequences makes it suitable for a wide range of time-series forecasting tasks. Vanilla LSTM, Stacked LSTM, and Bidirectional LSTM are different configurations of the LSTM architecture [49], [50].
- **GRU:** The GRU represents a variation of the LSTM architecture, prioritizing computational efficiency. It has been tailored to capture extended dependencies akin to LSTM while employing fewer parameters. This makes it well-suited for time-series forecasting applications where computational efficiency is a primary consideration. Additionally, within the GRU framework, one can find distinct configurations such as Vanilla GRU, Stacked GRU, and Bidirectional GRU [50].
- **ARIMA:** ARIMA stands as a venerable statistical model in time-series forecasting [51]. It methodically dissects time-series data into three integral components: autoregressive (AR), differencing (I), and moving average (MA). Determining the model's order parameters (p, d, q) becomes a crucial step in the process. ARIMA proves most effective when applied to stationary time-series data featuring evident trends and seasonality.
- **SARIMA:** SARIMA, an extension of ARIMA [51], effectively addresses seasonality in time-series data

by incorporating seasonal autoregressive (SAR), seasonal differencing (D), and seasonal moving average (SMA) components. It proves valuable when dealing with time-series data characterized by prominent and recurring seasonal patterns.

- **AutoRegression (AR):** A simple autoregressive model that anticipates future values by examining past values [52]. This model operates under the assumption that future values result from linear combinations of preceding values. To use this model effectively, one must carefully choose the lag order (p), indicating the number of past values to incorporate in the prediction.
- **Auto-Encoder:** An autoencoder is a neural network architecture employed for unsupervised feature learning and dimensionality reduction [53]. It can be customized for time-series forecasting by encoding historical time steps and decoding future predictions, making it particularly valuable for intricate and non-linear patterns found in time-series data.

F. PERFORMANCE METRICS

Our well-established performance metrics consist of classification and regression metrics.

1) CLASSIFICATION METRICS

There are different ways of measuring classification performance, such as accuracy, precision, recall, F1 score, and Mathew's correlation coefficient (MCC) [54], [55]. Accuracy tells us how close a measurement is to the actual value. The precision determines how relative the measured value is to the actual value. Recall defines the total number of positive results a machine learning model returns. Furthermore, we have calculated the testing and training accuracies. Testing and training accuracies are fundamental tools for understanding the model's performance, generalization capability, and potential issues like overfitting. A well-performing model should exhibit high testing accuracy while maintaining a reasonable balance with training accuracy.

$$Precision = \frac{TP}{TP + FP} \quad (25)$$

$$Recall = Sensitivity = \frac{TP}{TP + FN} \quad (26)$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (27)$$

$$F1\text{-score} = \frac{2 \times Recall \times Precision}{Recall + Precision} \quad (28)$$

Accuracy and F1 scores can be misleading because they need to fully account for the size of the four categories of the confusion matrix in the final score calculation. MCC is more informative than the F1 score and accuracy because it considers the balance of four confusion matrix categories (e.g., true positives, true negatives, false positives, and false

negatives).

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (29)$$

Here, we have captured the MCC values in % which can be converted to $[-1, 1]$ with a multiplication by 100.

2) REGRESSION METRICS

In time-series forecasting, different loss functions can be used depending on the nature of the problem and the desired properties of the anticipated values. In our time-series forecasting, we have used Root Mean Squared Error (RMSE), Forecast Error (FE), r^2 value, and residual standard deviation (RSD) [49]. MSE is a popular loss function that measures the average squared difference between the predicted and actual values. RMSE is the squared root of the MSE. It penalizes larger errors more heavily than smaller errors, making it sensitive to outliers. The RMSE loss function is defined as —

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (30)$$

$$RMSE = \sqrt{MSE} \quad (31)$$

The FE measures the discrepancy between the predicted values and the actual values in the time-series data. The R-squared value describes the percentage of the variance in the dependent variable (the actual values in the time-series) that is caused by the independent variable (the predicted values from the forecasting model). It gives an idea of how well the model's predictions match the actual data. The R-squared value is between 0 and 1. A value of 0 indicates that the model has no effect on the variability of the data, while a value of 1 indicates that the model is completely accurate. Intermediate values represent the percentage of variance that the model is capable of explaining.

G. HYPERPARAMETER TUNING PROCEDURES

To optimize the performance of ADSiamNet, we employed a systematic hyperparameter tuning approach. The following techniques were used:

- **Grid Search:** We utilized grid search strategy with early-stopping to determine optimal values for hyperparameters such as learning rate, batch size, dropout rate, and activation functions.
- **Learning Rate Schedule:** A dynamic learning rate was implemented using an exponential decay strategy to ensure stable convergence.
- **Batch Size Selection:** Empirical experiments were conducted to evaluate the impact of batch sizes (16, 32, 64) on model stability and convergence.
- **Dropout Rate and Activation Functions:** A dropout rate of 0.1 was selected to prevent overfitting, and the SeLU activation function was used for better self-normalizing properties.

- **Justification and Experimental Validation:** The selection of these hyperparameters was supported by ablation studies demonstrating their effects on model performance.

H. DATA NORMALIZATION AND HANDLING OF MISSING VALUES

To improve model stability and convergence, we applied the following preprocessing techniques:

- **Normalization:** We applied Min-Max Scaling and Z-score normalization to standardize the time-series data.
- **Handling Missing Values:** We explored three strategies—imputation using moving averages, removal of highly incomplete data points, and interpolation—choosing the best approach based on dataset characteristics.

I. DATASET SPLITS, TRAINING, AND VALIDATION DETAILS

To ensure robust evaluation, the dataset was partitioned as follows:

- **Training, Validation, and Test Split:** Data was split into 70% training, 15% validation, and 15% test sets.
- **Stratified Sampling:** Ensured balanced representation of normal and anomaly data points to prevent model bias.
- **Epochs and Early Stopping:** Training was conducted for 100 epochs with early stopping criteria based on validation loss.
- **Convergence Conditions:** Training was monitored using loss reduction trends to confirm stability.

V. EXPERIMENTS AND RESULTS

This section describes the setup we have used to run the overall experiment, experimental outcomes under the laboratory settings, key research findings, limitations, and the future scope.

A. EXPERIMENTAL SETUP

We used Python 3.8.5-supported language libraries such as pandas (v. 1.1.3), NumPy (v. 1.21.2), SciPy (v. 1.5.2), Matplotlib (v. 3.3.2), Seaborn (v. 0.11.0), Plotly (v. 5.2.1), scikit-learn or sklearn (v. 0.24.2), Keras (v. 2.6.0) and Graph Viz (v. 2.49.1) to process data, build and train deep learning models. We set up a Python environment on a Windows 10 operating system using the Anaconda distribution and installed Jupyter Notebook v. 6.4.5 for development, model analysis, and data visualization. The target system consists of 16 GB RAM and 64-bit architecture. Due to the medium size of the dataset, we used the CPU to run the experiments. For cross-verification of performance and resource sharing, we used the configuration of Google Colaboratory (colab).

B. EXPERIMENTAL RESULTS

1) ANOMALY DETECTION IN MOX2-5 AND ACTIGRAPHY DATASETS

We have worked on different window sizes, such as 25, 50, and 75 to prepare both datasets for anomaly detection

TABLE 3. Performance comparison of different optimizers for sliding window size = 25 for MOX2-5 activity datasets.

Optimizers	Precision	Recall	F1-score	Accuracy	MCC
Adam	85.00%	85.00%	85.0%	85.0%	83.85%
Adagrad	83.00%	83.00%	83.00%	83.00%	82.00%
RMSProp	82.00%	82.00%	82.00%	82.00%	80.20%
SGD	79.00%	79.00%	97.50%	79.00%	77.50%

TABLE 4. Performance comparison of different optimizers for sliding window size = 50 for MOX2-5 activity datasets.

Optimizers	Precision	Recall	F1-score	Accuracy	MCC
Adam	83.00%	83.00%	83.0%	83.0%	80.25%
Adagrad	82.00%	82.00%	82.00%	82.00%	79.60%
RMSProp	80.00%	80.00%	80.00%	80.00%	77.29%
SGD	77.00%	77.00%	77.00%	77.00%	74.50%

TABLE 5. Performance comparison of different optimizers for sliding window size = 75 for MOX2-5 activity datasets.

Optimizers	Precision	Recall	F1-score	Accuracy	MCC
Adam	81.00%	81.00%	81.0%	81.0%	79.55%
Adagrad	80.00%	80.00%	80.00%	80.00%	78.20%
RMSProp	78.00%	78.00%	78.00%	78.00%	75.26%
SGD	76.00%	76.00%	76.00%	76.00%	72.75%

and smoothing with ADSiamNet. For each sliding window size, we have captured the performance result of different optimizers, such as Adam, Adagrad, RMSProp, and SGD, in ADSiamNet (see Tables 3, 4, and 5 for MOX2-5 dataset, and Tables 6, 7, and 8 for actigraphy dataset). We have observed that step execution time increases with the growing window size, and it produces a slow model execution. Based on the overall performance, we have retained the Adam optimizer with a window size of 25 for anomaly detection and smoothing using ADSiamNet.

The Adam optimizer has produced accuracy on the training set of 84.0% and on the testing set of 85.6% for MOX2-5 dataset. Furthermore, the Adam optimizer has produced accuracy on the training set of 98.35% and on the testing set of 98.48% for actigraphy dataset. We calculated ADSiamNet model accuracy and model loss with window size 25 and Adam as an optimizer with a learning rate (α) of 0.0001. Moreover, we have used stride = 1, border = 0.5 (for accuracy checking in ADSiamNet), split_size = 0.2, batch_size = 128, and epochs = 200 to achieve the overall performance of the ADSiamNet model.

2) PERFORMANCE COMPARISON FOR ANOMALY DETECTION IN ACTIVITY DATASET

Here, we conducted a performance comparison between ADSiamNet and several well-established anomaly detection algorithms, including Isolation Forest, KNN, One-Class SVM, DTree, RF, Elliptic Envelope, MLP, 1D CNN, LSTM, and GRU. This evaluation was carried out using Activity

TABLE 6. Performance comparison of different optimizers for sliding window size = 25 for actigraphy activity datasets.

Optimizers	Precision	Recall	F1-score	Accuracy	MCC
Adam	98.00%	99.00%	98.5%	98.65%	97.65%
Adagrad	98.00%	98.00%	98.00%	98.00%	97.0%
RMSProp	98.00%	98.20%	98.10%	98.10%	97.2%
SGD	97.00%	98.00%	97.5%	97.5%	96.5%

TABLE 7. Performance comparison of different optimizers for sliding window size = 50 for actigraphy activity datasets.

Optimizers	Precision	Recall	F1-score	Accuracy	MCC
Adam	98.00%	98.20%	98.2%	98.2%	97.2%
Adagrad	97.00%	98.00%	97.50%	97.50%	97.1%
RMSProp	97.00%	98.00%	97.50%	97.50%	97.0%
SGD	97.00%	98.00%	97.50%	97.50%	97.0%

TABLE 8. Performance comparison of different optimizers for sliding window size = 75 for actigraphy activity datasets.

Optimizers	Precision	Recall	F1-score	Accuracy	MCC
Adam	98.00%	98.20%	98.1%	98.1%	97.0%
Adagrad	97.00%	98.00%	97.50%	97.50%	97.0%
RMSProp	97.00%	98.00%	97.50%	97.50%	97.0%
SGD	97.00%	98.00%	97.50%	97.50%	97.0%

TABLE 9. Performance comparison for anomaly detection in MOX2-5 activity dataset between ADSiamNet model for sliding window size = 25 and other machine learning and deep learning models.

Models	Precision	Recall	F1-score	Accuracy	MCC
ADSiamNet	85.00%	85.00%	85.0%	85.00%	83.35%
Isolation Forest	68.00%	68.00%	68.00%	68.00%	63.50%
KNN	61.00%	61.00%	61.00%	61.00%	54.20%
One-Class SVM	40.00%	40.00%	40.00%	40.00%	37.00%
Decision Tree	76.00%	76.00%	76.00%	76.00%	74.00%
Random Forest	65.00%	65.50%	65.50%	65.50%	63.00%
Elliptic Envelope	35.00%	35.00%	35.50%	35.00%	32.00%
MLP	77.00%	77.00%	77.00%	77.00%	74.00%
1D CNN	79.00%	79.00%	79.00%	79.00%	76.50%
LSTM	78.00%	78.00%	78.00%	78.00%	74.20%
GRU	78.00%	78.00%	78.00%	78.00%	74.50%

Datasets to showcase the effectiveness of the ADSiamNet model. The results are captured in Table 9 and Table 10 for MOX2-5 and actigraphy datasets, respectively. Overall, ADSiamNet has outperformed other models for both the activity datasets. The large volume of actigraphy datasets improved the performance of the ADSiamNet model. Therefore, large amounts of data have been crucial as they provided a more comprehensive representation of the underlying patterns and variations, leading to more accurate and generalizable predictions.

TABLE 10. Performance comparison for anomaly detection in actigraphy activity dataset between ADSiamNet model for sliding window size = 25 and other machine learning and deep learning models.

Models	Precision	Recall	F1-score	Accuracy	MCC
ADSiamNet	98.00%	99.00%	98.5%	98.65%	97.65%
Isolation Forest	94.00%	94.00%	94.00%	94.00%	93.00%
KNN	78.00%	79.00%	78.50%	78.50%	78.00%
One-Class SVM	50.00%	50.00%	50.00%	50.00%	49.00%
Decision Tree	94.00%	94.00%	94.00%	94.00%	90.00%
Random Forest	78.00%	78.50%	78.50%	78.50%	78.00%
Elliptic Envelope	48.00%	49.00%	49.50%	49.00%	48.00%
MLP	95.10%	95.10%	95.10%	95.10%	92.00%
1D CNN	95.00%	95.00%	95.50%	95.50%	93.50%
LSTM	94.00%	94.00%	94.00%	94.00%	92.20%
GRU	95.00%	95.00%	95.00%	95.00%	93.40%

3) COMPUTATIONAL COMPLEXITY ANALYSIS

In addition to detection performance, we also evaluated the computational complexity of ADSiamNet in terms of training and inference time across different window sizes and optimizers. The model exhibits **linear time complexity** $\mathcal{O}(n \cdot w \cdot k)$, where n is the number of time-series samples, w is the sliding window size, and k is the number of filters in the convolutional layers. This efficiency is attributed to the lightweight 1D convolutional operations and fixed-size input windows used during training and testing. On an Intel i7 CPU with 16GB RAM, the **average training time per epoch** for the MOX2-5 dataset (with window size 25) was approximately **1.85 seconds**, while the **inference time per sample** was around **0.002 seconds**. For the larger Actigraphy dataset, training required approximately **3.1 seconds per epoch**, with an average **inference time of 0.003 seconds per sample**. Compared to recurrent models such as LSTM and GRU, which involve sequential dependencies and exhibit higher time complexity $\mathcal{O}(n \cdot t^2)$ (where t is the sequence length), ADSiamNet provides superior runtime performance and reduced memory usage. Additionally, the pairwise comparison mechanism of the Siamese network is highly parallelizable and optimized, which enhances throughput and enables real-time or near-real-time anomaly detection. As the window size increases from 25 to 75, we observed that training time per epoch increases by approximately **35–40%**, which remains computationally tractable for practical applications. These results underscore ADSiamNet's suitability for real-time deployment in edge computing environments and wearable sensor-based systems.

4) COMPARING FORECASTING PERFORMANCES BETWEEN ANOMALOUS AND ANOMALY-SMOOTHED ACTIVITY DATASET

After the smoothing of anomalies from dataset D1, the smoothening approach has produced anomaly-smoothed dataset D2. Furthermore, we have applied different univariate

TABLE 11. Performances of different LSTM models on anomalous MOX2-5 dataset D1 of participant P1. MSE: Mean Squared Error, RMSE: Root Mean Squared Error, FE: Forecast Error, r2 (R-squared): the Coefficient of Determination, and RSD: Residual Standard Deviation.

Models	MSE	RMSE	FE	r2_value	RSD
Vanilla LSTM	19802.00	140.71	-4.30	0.60	36251.00
Stacked LSTM	18234.00	135.03	2.12	0.62	38229.00
Bi-Directional LSTM	17690.00	133.00	-0.39	0.66	37283.00
Vanilla GRU	17820.00	133.49	2.79	0.65	36260.00
Stacked GRU	17300.00	131.53	1.9	0.64	35524.00
Bi-Directional GRU	17721.00	133.12	-0.88	0.61	36653.00
SARIMA	28102.00	167.63	0.24	0.52	175.00
ARIMA	31754.00	178.19	0.42	0.40	199.88
Auto Regression	41200.00	202.97	110.08	-710.62	280.80
Auto-Encoder	14210.00	119.20	-0.08	0.60	160.88

time-series forecasting models on the datasets – D1 and D2, to compare the forecasting performances (see Table 11 and Table 12 for MOX2-5 dataset, and Table 13 and Table 14 for actigraphy dataset).

We have used 100 neurons to model the LSTMs and GRUs with `step_size = 3`, `optimizer = 'adam'`, `activation = 'relu'`, `loss = 'mse'`, `epochs = 200`, `batch_size = 50`, `validation_split = 0.05`. The Relu activation function improves computation and effectiveness in deep networks to prevent vanishing gradient problems. It sets all negative values to zero, keeping positive values unchanged, and introducing non-linearity. Furthermore, we have used “ReduceLROnPlateau” as a callback with `monitor = 'val_loss'`, `factor = 0.1`, `patience = 100`, to adjust the learning rate during training, based on the validation loss. In our SARIMA model, we specified the non-seasonal order as $(p, d, q) = (1, 1, 1)$ and the seasonal order as $(P, D, Q, S) = (1, 1, 1, 12)$. The ARIMA model parameters were defined as $(2, 1, 1)$.

For both AutoRegression and Auto-Encoder, we utilized a window size of 5. Our Auto-Encoder model comprises four Dense layers with a sequence of neurons: 16, 8, 16, and 5, with ReLU activation applied at each layer except the final one. The model was compiled using the “ADAM” optimization function with Mean Squared Error (MSE) as the loss function. This autoencoder architecture follows a relatively simple and symmetrical design. It first reduces the input data into a lower-dimensional representation (encoding) through the initial two layers and subsequently reconstructs the original data through the final two layers. The primary objective is to acquire a concise representation of the input data that effectively captures its fundamental features.

The findings indicate that the forecasting performance significantly improves when using an anomaly-smoothed

TABLE 12. Performances of different LSTM models on anomaly-smoothed MOX2-5 dataset D2 of participant P1. MSE: Mean Squared Error, RMSE: Root Mean Squared Error, FE: Forecast Error, r2 (R-squared): the Coefficient of Determination, and RSD: Residual Standard Deviation.

Models	MSE	RMSE	FE	r2_value	RSD
Vanilla LSTM	0.401	0.633	0.141	1.00	29375.00
Stacked LSTM	1.358	1.165	0.725	1.00	27920.00
Bi-Directional LSTM	1.238	1.112	0.697	1.00	26921.00
Vanilla GRU	40.22	6.342	1.89	0.99	30328.00
Stacked GRU	32.56	5.70	0.51	0.99	29724.00
Bi-Directional GRU	32.44	5.69	-1.25	0.99	29276.00
SARIMA	52.06	7.22	0.124	0.99	7.40
ARIMA	51.87	7.20	0.113	0.99	7.10
Auto Regression	28440.00	168.64	95.60	-134.23	165.00
Auto-Encoder	24.62	4.96	0.07	0.99	5.89

TABLE 13. Performances of different LSTM models on anomalous actigraphy dataset D1 of participant P1. MSE: Mean Squared Error, RMSE: Root Mean Squared Error, FE: Forecast Error, r2 (R-squared): the Coefficient of Determination, and RSD: Residual Standard Deviation.

Models	MSE	RMSE	FE	r2_value	RSD
Vanilla LSTM	17612.28	132.71	-3.29	0.61	34639.20
Stacked LSTM	17634.21	132.8	1.82	0.64	35219.49
Bi-Directional LSTM	17744.70	133.21	-0.40	0.62	34883.82
Vanilla GRU	17776.06	133.33	2.18	0.63	35160.43
Stacked GRU	17604.12	132.68	1.47	0.63	35084.94
Bi-Directional GRU	17651.33	132.86	-0.93	0.62	34843.99
SARIMA	29113.56	170.62	0.19	0.49	170.64
ARIMA	28854.22	169.86	0.22	0.49	169.88
Auto Regression	36210.55	190.29	87.05	-411.88	190.30
Auto-Encoder	13310.86	115.37	-0.05	0.53	115.38

dataset (D2). In the context of the analyzed activity dataset, the Auto-Encoder-based forecasting method outperforms others for both D1 and D2. Notably, the overall forecasting performance exhibits significant enhancement in D2, providing substantial empirical support for our initial hypothesis.

5) VERIFICATION OF ADSiamNet MODEL ON THE NAB BENCHMARK DATASET FOR ANOMALY DETECTION

To cross-verify the performance of ADSiamNet model on anomaly detection on different domains, we have applied the model to the various datasets which are stored in the following folders in NAB — artificialNoAnomaly, artificial-

TABLE 14. Performances of different LSTM models on anomaly-smoothed actigraphy dataset D2 of participant P1. MSE: Mean Squared Error, RMSE: Root Mean Squared Error, FE: Forecast Error, r2 (R-squared): the Coefficient of Determination, and RSD: Residual Standard Deviation.

Models	MSE	RMSE	FE	r2_value	RSD
Vanilla LSTM	0.382	0.618	0.119	1.00	26775.50
Stacked LSTM	1.228	1.108	0.613	1.00	26817.68
Bi-Directional LSTM	1.868	1.367	0.795	1.00	26834.84
Vanilla GRU	30.56	5.52	1.04	0.99	28368.58
Stacked GRU	27.59	5.25	0.38	0.99	28314.78
Bi-Directional GRU	31.014	5.56	-1.25	0.99	28177.8
SARIMA	46.36	6.80	0.004	0.99	6.80
ARIMA	46.27	6.80	0.003	0.99	6.80
Auto Regression	24240.61	155.69	90.30	-124.13	155.7
Auto-Encoder	21.52	4.64	0.02	0.99	4.63

TABLE 15. ADSiamNet on datasets in the artificialNoAnomaly folder.

Numenta Datasets	Precision	Recall	F1-score	Accuracy	MCC
art daily no noise	100.00%	100.00%	100.00%	100.00%	100.00%
art daily perfect square wave	100.00%	100.00%	100.00%	100.00%	100.00%
art daily small noise	100.00%	100.00%	100.00%	100.00%	100.00%
art flat-line	100.00%	100.00%	100.00%	100.00%	100.00%
art noisy	100.00%	100.00%	100.00%	100.00%	100.00%

WithAnomaly, realAdExchange, realAWScloudwatch, realKnownCause, realTraffic, and realTweets. Existing anomaly detectors in the NAB datasets follow an unsupervised approach. However, this is the first study with a deep learning method to perform anomaly detection in the NAB datasets using a supervised approach. The performances of our proposed ADSiamNet model have been captured in Tables 15, 16, 17, 18, 19, 20, and 21 for individual datasets in the respective folders. Overall, the ADSiamNet has performed better than the existing anomaly detectors in the NAB datasets (see [18]). All the datasets in the artificialNoAnomaly folder and the ec2_cpu_utilization_c6585a data in the realAWScloudwatch folder are anomaly free; therefore, ADSiamNet has achieved a training and testing accuracy score of 100% which is justified and free from the overfitting.

TABLE 16. ADSiamNet on datasets in the artificialWithAnomaly folder.

Numenta Datasets	Precision	Recall	F1-score	Accuracy	MCC
art_daily_flatmiddle	94.00%	95.00%	94.50%	94.70%	94.00%
art_daily_jumpsdown	93.00%	93.50%	93.25%	93.25%	93.00%
art_daily_jumpsup	92.00%	93.00%	92.50%	92.26%	92.00%
art_daily_nojump	89.00%	89.00%	89.00%	89.10%	89.00%
art_increase_spike_density	89.00%	89.00%	89.00%	89.40%	89.00%
art_load_balancer_spikes	90.00%	90.00%	90.00%	90.14%	90.00%

TABLE 17. ADSiamNet on datasets in the realAdExchange folder.

Numenta Datasets	Precision	Recall	F1-score	Accuracy	MCC
exchange-2_cpc_results	88.00%	89.00%	88.50%	88.82%	88.00%
exchange-2_cpm_results	86.00%	87.50%	86.50%	86.47%	86.00%
exchange-3_cpc_results	86.00%	87.50%	86.50%	86.50%	86.00%
exchange-3_cpm_results	88.00%	88.00%	88.00%	88.10%	88.00%
exchange-4_cpc_results	84.00%	85.00%	84.50%	84.80%	84.00%
exchange-4_cpm_results	82.00%	83.00%	82.50%	82.23%	82.00%

C. DISCUSSION

The ADSiamNet-based approach for anomaly detection has been an effective, novel, and innovative approach that leverages the Siamese architecture and CNNs for detecting and smoothing anomalies in sequential data. By using this architecture, the network learns to compare input sequences and recognize abnormal patterns without requiring numerous labeled anomaly samples. On the used activity dataset (D1), our ADSiamNet model has effectively distinguished anomalies from normal instances with 98.65% accuracy (98.00% precision, 99.0% recall, 98.5% F1-score, and 97.65% MCC) in publicly available actigraphy dataset and 85.0% accuracy (85.00% precision, 85.0% recall, 85.0% F1-score, and 83.85% MCC) in real-time collected MOX2-5 activity dataset.

The discrepancy in accuracy between the two datasets in the anomaly detection task using the ADSiamNet model can be attributed to several factors. The larger dataset, comprised of public actigraphy time-series step data, offers the model a wealth of instances to learn from, enabling it

TABLE 18. ADSiamNet on datasets in the realAWSCloudwatch folder.

Numenta Datasets	Precision	Recall	F1-score	Accuracy	MCC
ec2_cpu_utilization_5f5533	88.00%	89.00%	88.50%	89.05%	88.00%
ec2_cpu_utilization_24ae8d	88.00%	89.50%	88.50%	88.86%	88.00%
ec2_cpu_utilization_53ea38	88.00%	89.50%	88.50%	88.74%	88.00%
ec2_cpu_utilization_77c1ca	89.00%	90.00%	89.50%	89.73%	88.00%
ec2_cpu_utilization_825cc2	91.00%	92.00%	91.50%	91.51%	90.00%
ec2_cpu_utilization_ac20cd	88.00%	90.00%	89.00%	89.67%	89.00%
ec2_cpu_utilization_c6585a	100.00%	100.00%	100.00%	100.00%	100.00%
ec2_cpu_utilization_fe7f93	89.00%	90.00%	89.50%	89.39%	89.00%
ec2_disk_write_bytes_1ef3de	89.00%	90.00%	89.50%	89.24%	89.00%
ec2_disk_write_bytes_c0d644	88.00%	88.00%	88.00%	88.24%	88.00%
ec2_network_in_5abac7	88.00%	89.00%	88.50%	88.82%	88.00%
ec2_network_in_257a54	89.00%	89.50%	89.00%	89.24%	88.00%
elb_request_count_8c0756	91.00%	91.00%	91.00%	91.00%	91.00%
grok_asg_anomaly	88.00%	89.00%	88.50%	88.33%	88.00%
iio_us-east-1_i_a2eb1cd9_NetworkIn	84.00%	85.00%	84.50%	84.80%	84.00%
rds_cpu_utilization_cc0c53	86.00%	87.00%	86.50%	86.65%	86.00%
rds_cpu_utilization_e47b3b	89.00%	89.50%	89.25%	89.25%	89.00%

to capture a wider array of patterns and variations in the data. In contrast, the smaller dataset, consisting of real-time MOX2-5 time-series step data, lacks the diversity

TABLE 19. ADSiamNet on datasets in the realKnownCause folder.

Numenta Datasets	Precision	Recall	F1-score	Accuracy	MCC
ambient temperature system failure	89.00%	89.50%	89.00%	89.38%	89.00%
cpu utilization asg misconfiguration	92.00%	93.8%	93.50%	93.64%	92.00%
ec2 request latency system failure	90.00%	91.00%	90.50%	90.50%	89.00%
machine temperature system failure	90.00%	90.50%	90.00%	90.10%	90.00%
nyc taxi	88.00%	89.00%	88.50%	88.5%	88.00%
rogue agent key hold	86.00%	87.00%	86.50%	86.94%	86.00%
rogue agent key updown	88.00%	89.00%	88.50%	88.94%	88.00%

TABLE 20. ADSiamNet on datasets in the realTraffic folder.

Numenta Datasets	Precision	Recall	F1-score	Accuracy	MCC
occupancy 6005	90.00%	91.50%	91.50%	91.66%	90.00%
occupancy t4013	89.00%	90.00%	89.50%	89.75%	89.00%
speed 6005	90.00%	91.00%	90.50%	90.91%	90.00%
speed 7578	86.00%	86.50%	86.00%	86.04%	85.00%
speed t4013	89.00%	89.00%	89.00%	89.32%	89.00%
TravelTime 387	87.00%	87.00%	87.00%	87.07%	86.00%
TravelTime 451	88.00%	89.00%	88.50%	88.88%	88.00%

TABLE 21. ADSiamNet on datasets in the realTweets folder.

Twitter Numenta Datasets	Precision	Recall	F1-score	Accuracy	MCC
AAPL	90.00%	91.00%	90.50%	90.22%	89.00%
AMZN	90.00%	91.00%	90.50%	90.42%	89.00%
CRM	90.00%	91.00%	90.50%	90.68%	90.00%
CVS	90.00%	90.50%	90.00%	90.33%	90.00%
FB	90.00%	91.00%	90.50%	90.58%	89.00%
GOOG	92.00%	92.00%	92.00%	92.16%	92.00%
IBM	89.00%	90.00%	89.50%	89.85%	88.00%
KO	90.00%	91.00%	90.50%	90.51%	90.00%
PFE	90.00%	91.00%	90.50%	90.28%	90.00%
UPS	90.00%	90.00%	90.00%	90.08%	90.00%

necessary for the model to generalize effectively to unseen instances. Additionally, differences in the representation of the time-series data between the two datasets may play a role, with the larger dataset potentially benefitting from more informative features or better preprocessing techniques. Furthermore, in the smaller dataset, the model may memorize the training data rather than extracting underlying patterns, resulting in decreased accuracy on unseen data. Lastly, variations in data quality could contribute to the accuracy gap,

as the larger dataset may exhibit higher data quality, while the smaller dataset may contain more noise or outliers, posing challenges for the model in distinguishing between normal and anomalous patterns.

In addressing the accuracy discrepancy, potential future research avenues include exploring data augmentation techniques to expand the smaller dataset by introducing variations such as noise addition, time-series shifting, or scaling. Ensemble methods, combining predictions from diverse models or architectures, could enhance overall performance and resilience. Additionally, transfer learning offers promise by pretraining models on the larger dataset and fine-tuning on the smaller dataset, capitalizing on knowledge gleaned from the former to potentially boost performance on the latter. These avenues collectively present opportunities for enhancing accuracy and robustness in anomaly detection tasks.

Furthermore, the anomaly smoothing in time-series (D1) with a smoothing approach, also helps to improve the prediction efficiency of the dataset (D2) with the reduction of forecasting losses, such as MSE: 21.52, RMSE: 4.64, FE: 0.02, r_2 _score: 0.99, and RSD: 4.63 with the Auto-Encoder model (epoch = 250 and batch size = 32). For the benchmark testing of ADSiamNet model, we have used the NAB anomalous and anomaly-free datasets. The NAB scores are normalized [56] such that the maximum possible value is 100.0 (i.e., a perfect detector), and the baseline of 0.0 is determined by a “null” detector (or no detection). Additionally, we tested our proposed ADSiamNet model on the NAB dataset. In this context, ADSiamNet demonstrated superior performance when compared to other anomaly detection methods featured in the NAB repository. Our ADSiamNet model has achieved a mean accuracy score of 90.43% (a standard deviation of 3.8911) on the NAB anomaly detection dataset, and the achieved mean accuracy score is better than the other anomaly detection algorithm on the NAB datasets – ARTime (74.9%), Numenta HTM* (70.5%-69.7%), CAD OSE[†] (69.9%), earthgecko Skyline (58.2%), KNN CAD[†] (58.0%), Relative Entropy (54.6%), Random Cut Forest**** (51.7%), Twitter ADVec v1.0.0 (47.1%), Windowed Gaussian (39.6%), Etsy Skyline (35.7%), Bayesian Changepoint** (17.7%), EXPoSE (16.4%), Random*** (11.0%), Numenta HTMusing NuPIC v0.5.6* (70.1%), nab-comportex[†] (64.6%), NumentaTM HTM* (64.6%), HTM Java (56.8%), and Numenta HTM* no likelihood (53.62%).

In the ADSiamNet model, the 1D CNN’s convolutional layers are adept at processing sequential data, effectively capturing local patterns and correlations within input sequences. This capability proves particularly advantageous for identifying anomalies that exhibit distinctive patterns within short data windows. Additionally, ADSiamNet excels at learning meaningful feature representations and embeddings for both normal and anomalous sequences. This empowers the network to gauge sequence similarity in the embedding space, rendering it effective for anomaly detection. The employment of metric learning through the Siamese network enhances

the discrimination between normal and abnormal sequences. Consequently, the proposed ADSiamNet model is well-suited for time-series data, including sensor measurements and any sequential data where anomalies manifest as deviations from established patterns over time. Moreover, ADSiamNet's performance on NAB datasets underscores its promise for anomaly detection across diverse univariate datasets spanning various domains. Furthermore, the SeLU activation function within the ADSiamNet model plays a pivotal role in normalizing activation distributions during training. This effectively mitigates issues such as vanishing and exploding gradients that can plague traditional activation functions like sigmoid and tanh. SeLU's inherent self-normalizing property helps maintain a mean activation close to zero and unit variance, making it particularly well-suited for the ADSiamNet model. Additionally, SeLU introduces an implicit regularization effect on the model due to its distinctive shape and scale characteristics. It can also facilitate automatic adjustment of the learning rate (α) parameter during training, promoting faster convergence. However, it's essential to note that SeLU's effectiveness may vary depending on factors such as the data's nature, network design, and the specific problem being tackled. Its advantages may not be universal and can be influenced by these contextual factors.

1) ADVANTAGES OVER CHALLENGES

While the 1D CNN-based ADSiamNet presents notable advantages in univariate anomaly detection, it also encounters several challenges. One significant challenge lies in its capacity to capture long-term dependencies within a limited volume of time-series data adequately. Due to the nature of 1D CNNs, which are primarily designed for identifying local and short-term patterns, they may struggle to discern anomalies that manifest as subtle deviations over extended periods. This limitation becomes particularly pronounced when anomalies are characterized by their relation to long-term trends or patterns in the limited volume of data. Additionally, the effectiveness of the ADSiamNet model may be contingent upon the careful selection and tuning of hyperparameters, such as window size and threshold values, to ensure optimal performance. Overall, while the 1D CNN-based ADSiamNet offers promising capabilities for univariate anomaly detection, addressing these challenges is essential to enhance its efficacy and applicability in practical settings.

The proposed ADSiamNet model offers the following theoretical and practical advantages over existing approaches in the realm of time-series anomaly detection, – **a. Theoretical Superiority:** The integration of Siamese networks enhances the model's ability to discern intricate patterns and dependencies within time-series data, leading to a more nuanced understanding of the underlying dynamics and improved accuracy in anomaly detection. Additionally, the utilization of a quantile-based approach for anomaly smoothing enables the model to mitigate the impact of anomalies on data analysis while preserving essential trends and patterns

within the time-series, thereby enhancing the robustness and reliability of anomaly detection outcomes. **b. Practical Superiority:** In practical applications, the proposed model demonstrates superiority in accuracy and precision through the integration of Siamese networks and quantile-based smoothing, enabling precise anomaly detection and fostering improved decision-making processes. Additionally, its linear computational complexity ensures scalability to large datasets while maintaining efficiency, making it suitable for real-time or near-real-time scenarios. Moreover, the model's adaptable architecture allows for tailoring and fine-tuning to accommodate diverse datasets and application domains, enhancing its applicability and flexibility across various contexts. **c. Broader Application Domain:** ADSiamNet has significant potential for real-world applications, particularly in predictive maintenance, fraud detection, and condition monitoring. In industrial systems, it can identify early indicators of failure, reducing downtime and maintenance costs. Its real-time processing capability also supports monitoring high-frequency sensor data. In finance, ADSiamNet can detect fraudulent transactions by identifying subtle deviations from normal patterns. In healthcare, it can analyze physiological signals from wearable devices, enabling timely medical interventions. Its lightweight design and low-latency performance make it suitable for IoT-enabled devices and edge computing environments, providing utility in resource-constrained scenarios. These capabilities also position it for applications in cybersecurity, environmental monitoring, and autonomous systems.

Overall, the proposed ADSiamNet model represents a significant advancement in time-series anomaly detection by leveraging state-of-the-art methodologies and innovative techniques. Its theoretical foundations and practical implementations contribute to its superiority over existing approaches, offering enhanced accuracy, scalability, adaptability, and efficiency in anomaly detection tasks.

2) SCALABILITY OF ADSIAMNET

ADSiamNet's scalability has been validated on NAB benchmark datasets of varying sizes from different domains, and a synthetic univariate time-series data with anomalies of 1 million time-step, demonstrating low inference latency averaging less than 5 milliseconds per sample. The 1D CNN-based architecture leverages parallel processing, unlike traditional supervised methods such as Isolation Forest, KNN, and Decision Trees, which require sequential processing and are computationally intensive. The 1D CNN-based model has proven to be a better choice than LSTM and GRU for anomaly detection in 1D time series data due to its ability to efficiently capture local temporal patterns with lower computational complexity, allowing for faster training and real-time processing. The 1D CNN-based lightweight model structure supports batch processing, enabling efficient processing of large datasets without sacrificing accuracy. Additionally, ADSiamNet's compatibility can make it suitable for real-time monitoring applications in domains such

as industrial automation, healthcare, and network security, where fast anomaly detection is crucial.

3) QUANTILE-BASED SMOOTHING

Quantile smoothing has been chosen for anomaly rectification due to its robustness to outliers and its ability to preserve statistical patterns without introducing biases. Unlike interpolation, which assumes continuity, quantile smoothing preserves temporal structures, making it ideal for time series data. It avoids the assumptions and prediction errors associated with data imputation methods, ensuring that trends and seasonal variations are preserved. This approach improves anomaly rectification without distorting the data distribution, increasing the model's ability to effectively detect anomalies.

4) COMPARING WITH OTHER SUPERVISED ANOMALY DETECTION METHODS

The proposed ADSiamNet model demonstrates high computational efficiency by relying on 1D CNNs. The architecture is lightweight, requires fewer parameters, and is faster to train compared to traditional supervised methods such as Isolation Forest, KNN, One-SVM, DTree, RF, Elliptic Envelope, and MLP. These methods often struggle with scalability and computational overhead when handling large datasets. LSTMs are well suited for scenarios requiring modeling of complex temporal relationships, but tend to require more computational resources due to their recursive structure and memory-based mechanisms. GRUs offer a more simplified alternative to LSTMs, providing faster training and reduced computational load while retaining most of the capabilities needed for sequential learning tasks. This balance between performance and efficiency makes GRUs ideal for applications requiring fast processing without significant loss of modeling accuracy. In contrast, ADSiamNet processes data in parallel through convolution layers, enabling faster training and inference. Its ability to efficiently capture temporal dependencies makes it suitable for real-time and large-scale anomaly detection tasks. Experimental results show that ADSiamNet converges quickly, achieves low memory consumption, and maintains low inference latency, making it ideal for resource-constrained environments and real-time applications.

5) FUTURE WORK

The ADSiamNet model has shown strong efficacy in univariate anomaly detection but could be enhanced by expanding its scope to multivariate time-series data, common in finance, healthcare, and network monitoring. Adapting the model to integrate and analyze multiple data streams simultaneously will broaden its applicability and improve its utility in complex scenarios.

Extending ADSiamNet to high-dimensional data will introduce challenges, primarily in managing dependencies and correlations between multiple variables, as well as

scaling computational complexity. Traditional methods such as Isolation Forest and Random Forest inherently do not capture dependencies between variables, while ADSiamNet can be improved to handle such relationships. Future work will explore the integration of attention mechanisms and graph-based representations to efficiently model correlations. Techniques such as multi-channel convolution layers will also be explored to preserve computational efficiency while scaling to high-dimensional data. These improvements aim to extend the capabilities of the model without compromising its speed and scalability.

Optimizing the model's code execution time is essential as data complexity increases, particularly for real-time applications. Future efforts will explore integrating fast spatial-temporal information compression techniques to improve efficiency and throughput, inspired by advancements in real-time traffic flow forecasting. Experimenting with multivariate datasets will be vital to assess the model's capability to handle diverse conditions and interactions. This will help refine the ADSiamNet's architecture and training process for better performance across different domains. Concerns about potential comparison unfairness due to varying sliding window sizes are also valid. Ensuring that the window size used with ADSiamNet is consistent with other models during testing will maintain fairness and validity in comparative evaluations.

Transformer models [69], [70] have shown exceptional performance in sequential data modeling. However, challenges such as quadratic complexity in self-attention and large-scale training requirements make them less feasible for real-time anomaly detection. Future work will explore optimized transformer variants (e.g., time-series transformers [69], [70]) to mitigate these issues. Furthermore, Generative adversarial networks (GANs) [71], [72] are widely used for anomaly detection by learning normal distributions of data. However, they suffer from problems such as mode collapse and instability during training. Future work will explore conditional GANs and self-supervised learning frameworks to improve robustness. Furthermore, GANs are widely used for anomaly detection by learning normal distributions of data. However, they suffer from problems such as mode collapse and instability during training. Future work will explore conditional GANs and self-supervised learning frameworks to improve robustness.

Anomalies in time-series data usually evolve over time, which makes static models less effective. Future research will integrate adaptive learning techniques, such as online learning and reinforcement-based anomaly adaptation, to ensure the robustness of models in dynamic environments. Anomaly detection datasets are often imbalanced, with significantly fewer anomaly instances than normal data. To address this, we will explore advanced resampling techniques such as Synthetic Minority Over-sampling (SMOTE) [73], as well as augmentation techniques [45] adapted to time-series data to provide balanced training distributions.

By addressing these limitations and implementing the suggested improvements, the ADSiamNet model can be developed more robustly, enhancing its performance and applicability in real-world settings and diverse datasets incorporating anomaly detection and prediction in a chaotic time-series. Moreover, in future, recent advancements such as deep learning models like LSTM, GANs and the latest techniques in transfer learning can be tested for integration in ADSiamNet model to make it robust and efficient.

VI. CONCLUSION

Detecting anomalies in time-series data is a complex undertaking that demands a thoughtful approach, considering the unique characteristics of the data, domain expertise, and specific objectives. The field of detecting anomalous behavior in time-series data is dynamic, with continuous efforts to enhance accuracy and efficiency across various applications. This study introduces an innovating approach for detecting anomalies in univariate time-series data, employing the ADSiamNet model. The proposed ADSiamNet model plays a pivotal role by learning a similarity function between inputs, typically presented as pairs or tuples. The optimization process employs contrastive loss to effectively map data points into an embedding space, facilitating the clear distinction between normal and anomalous instances. As a result, the ADSiamNet model demonstrates impressive accuracies in identifying anomalies during inference for the identified publicly available actigraphy dataset and real-time collected MOX2-5 dataset, and outperforms other supervised anomaly detection models. Moreover, It captures competitive accuracies in the NAB benchmark datasets. Additionally, this study addresses anomaly smoothing within time-series data by employing a smoothing mechanism to reduce the impact of anomalies based on the quantile approach. This approach not only enhances the forecasting efficiency of the dataset but also reduces forecasting errors. It's essential to note that this study specifically focuses on univariate time-series anomaly detection. Future research endeavors can extend this approach to multivariate time-series data. Leveraging multivariate configurations of deep learning time-series regression models from existing libraries can be an effective strategy. Moreover, following anomaly detection using the Siamese network in multi-variate time-series, integrating an autoencoder for anomaly smoothing is a promising avenue for future investigation, though it falls outside the scope of this study.

AUTHOR CONTRIBUTIONS STATEMENT

Ayan Chatterjee: Conceptualization, Ideation, Methodology, Analysis, Experimentation, Writing—original draft, writing—review, and editing. Vajira Thambawita: Conceptualization, Ideation, and Analysis, writing—review and editing. Michael A. Riegler: Conceptualization, writing—review and editing. Pål Halvorsen: Writing—review and editing. All authors reviewed and made necessary improvements to the article to ensure its quality.

DATA AVAILABILITY

The author Ayan Chatterjee can be contacted for the data and the GitHub repository will be made public.

COMPETING INTERESTS

The authors declare no competing interests.

ACKNOWLEDGMENT

The authors acknowledge the Simula Metropolitan Center for Digital Engineering (SimulaMet), Norway; the University of Agder; NILU; and OsloMet, Norway, to carry out this research.

REFERENCES

- [1] J. D. Hamilton, *Time Series Analysis*. Princeton, NJ, USA: Princeton Univ. Press, 2020.
- [2] J. D. Cryer, *Time Series Analysis*, vol. 286. Boston, MA, USA: Duxbury Press, 1986.
- [3] R. S. Tsay, *Analysis of Financial Time Series*. Hoboken, NJ, USA: Wiley, 2005.
- [4] M. Nerlove, D. M. Grether, and J. L. Carvalho, *Analysis of Economic Time Series: A Synthesis*. New York, NY, USA: Academic, 2014.
- [5] S. L. Zeger, R. Irizarry, and R. D. Peng, "On time series analysis of public health and biomedical data," *Annu. Rev. Public Health*, vol. 27, no. 1, pp. 57–79, Apr. 2006.
- [6] N. Wiener, N. Wiener, C. Mathematician, N. Wiener, N. Wiener, and C. Mathématicien, *Extrapolation, Interpolation, and Smoothing of Stationary Time Series: With Engineering Applications*, vol. 113. Cambridge, MA, USA: MIT Press, 1949.
- [7] P. J. Brockwell and R. A. Davis, *Time Series: Theory and Methods*. Cham, Switzerland: Springer, 2009.
- [8] M. Hu, Z. Ji, K. Yan, Y. Guo, X. Feng, J. Gong, X. Zhao, and L. Dong, "Detecting anomalies in time series data via a meta-feature based approach," *IEEE Access*, vol. 6, pp. 27760–27776, 2018.
- [9] A. Blázquez-García, A. Conde, U. Mori, and J. A. Lozano, "A review on outlier/anomaly detection in time series data," *ACM Comput. Surveys*, vol. 54, no. 3, pp. 1–33, Apr. 2022.
- [10] R. Foorthuis, "On the nature and types of anomalies: A review of deviations in data," *Int. J. Data Sci. Anal.*, vol. 12, no. 4, pp. 297–331, Oct. 2021.
- [11] R. Foorthuis, "A typology of data anomalies," in *Proc. Int. Conf. Inf. Process. Manage. Uncertainty Knowl.-Based Syst.* Cham, Switzerland: Springer, May 2018, pp. 26–38.
- [12] D. Samariya and A. Thakkar, "A comprehensive survey of anomaly detection algorithms," *Ann. Data Sci.*, vol. 10, no. 3, pp. 829–850, Nov. 2021.
- [13] M. Kloska, G. Grmanova, and V. Rozinajova, "Expert enhanced dynamic time warping based anomaly detection," *Expert Syst. Appl.*, vol. 225, Sep. 2023, Art. no. 120030.
- [14] T. Zoppi and A. Ceccarelli, "Prepare for trouble and make it double! supervised-unsupervised stacking for anomaly-based intrusion detection," *J. Netw. Comput. Appl.*, vol. 189, Sep. 2021, Art. no. 103106.
- [15] B. Ramachandra, M. J. Jones, and R. Raju Vatsavai, "Learning a distance function with a Siamese network to localize anomalies in videos," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Mar. 2020, pp. 2587–2596.
- [16] I. Melekhov, J. Kannala, and E. Rahtu, "Siamese network features for image matching," in *Proc. 23rd Int. Conf. Pattern Recognit. (ICPR)*, Dec. 2016, pp. 378–383.
- [17] K. Martin, N. Wiratunga, S. Sani, and S. Massie, "A convolutional Siamese net-work for developing similarity knowledge in the selfback dataset," in *Proc. CEUR Workshop*, 2017, pp. 85–94.
- [18] NAB: The Numenta Anomaly Benchmark (NAB). Accessed: Jun. 30, 2024. [Online]. Available: <https://github.com/numenta/NAB>
- [19] Z. Ji, Y. Wang, K. Yan, X. Xie, Y. Xiang, and J. Huang, "A space-embedding strategy for anomaly detection in multivariate time series," *Expert Syst. Appl.*, vol. 206, Nov. 2022, Art. no. 117892.
- [20] Y. Wang, H. Wu, J. Dong, Y. Liu, M. Long, and J. Wang, "Deep time series models: A comprehensive survey and benchmark," 2024, *arXiv:2407.13278*.

- [21] S. Reza, M. C. Ferreira, J. J. M. Machado, and J. M. R. S. Tavares, "Traffic state prediction using one-dimensional convolution neural networks and long short-term memory," *Appl. Sci.*, vol. 12, no. 10, p. 5149, May 2022.
- [22] W. Chen, W. Hussain, F. Cauteruccio, and X. Zhang, "Deep learning for financial time series prediction: A state-of-the-art review of standalone and hybrid models," *CMES-Comput. Model. Eng. Sci.*, vol. 139, no. 1, pp. 187–224, Dec. 2023.
- [23] M. Hu, X. Feng, Z. Ji, K. Yan, and S. Zhou, "A novel computational approach for discord search with local recurrence rates in multivariate time series," *Inf. Sci.*, vol. 477, pp. 220–233, Mar. 2019.
- [24] S. Kanarachos, S.-R.-G. Christopoulos, A. Chroneos, and M. E. Fitzpatrick, "Detecting anomalies in time series data via a deep learning algorithm combining wavelets, neural networks and Hilbert transform," *Expert Syst. Appl.*, vol. 85, pp. 292–304, Nov. 2017.
- [25] K. Shaukat, T. M. Alam, S. Luo, S. Shabbir, I. A. Hameed, J. Li, S. K. Abbas, and U. Javed, "A review of time-series anomaly detection techniques: A step to future perspectives," in *Proc. Future Inf. Commun. Conf.*, vol. 1. Cham, Switzerland: Springer, 2021, pp. 865–877.
- [26] M. Grill, T. Pevný, and M. Rehak, "Reducing false positives of network anomaly detection by local adaptive multivariate smoothing," *J. Comput. Syst. Sci.*, vol. 83, no. 1, pp. 43–57, Feb. 2017.
- [27] D. Ostroski, K. Slovenec, I. Brajdic, and M. Mikuc, "Anomaly correction in time series data for improved forecasting," in *Proc. 16th Int. Conf. Telecommun. (ConTEL)*, Jun. 2021, pp. 85–88.
- [28] N. Goernitz, M. Kloft, K. Rieck, and U. Brefeld, "Toward supervised anomaly detection," *J. Artif. Intell. Res.*, vol. 46, pp. 235–262, Feb. 2013.
- [29] J. Ma, L. Sun, H. Wang, Y. Zhang, and U. Aickelin, "Supervised anomaly detection in uncertain pseudoperiodic data streams," *ACM Trans. Internet Technol.*, vol. 16, no. 1, pp. 1–20, Feb. 2016.
- [30] J. G. Fornás, E. H. Jaraba, and A. L. Estopiñán, "TransSiamese: A transformer-based Siamese network for anomaly detection in time series as approach for fault location in distribution grids," *IEEE Access*, vol. 11, pp. 103431–103451, 2023.
- [31] M. E. Villa-Pérez, M. Á. Álvarez-Carmona, O. Loyola-González, M. A. Medina-Pérez, J. C. Velasco-Rossell, and K.-K.-R. Choo, "Semi-supervised anomaly detection algorithms: A comparative summary and future research directions," *Knowl.-Based Syst.*, vol. 218, Apr. 2021, Art. no. 106878.
- [32] M. Goldstein and S. Uchida, "A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data," *PLoS ONE*, vol. 11, no. 4, Apr. 2016, Art. no. e0152173.
- [33] M. Munir, S. A. Siddiqui, A. Dengel, and S. Ahmed, "DeepAnT: A deep learning approach for unsupervised anomaly detection in time series," *IEEE Access*, vol. 7, pp. 1991–2005, 2019.
- [34] H. Xiao, D. Guan, R. Zhao, W. Yuan, Y. Tu, and A. M. Khattak, "Semi-supervised time series anomaly detection model based on LSTM autoencoder," in *Proc. Int. Conf. Big Data Secur.* Singapore: Springer, Dec. 2020, pp. 41–53.
- [35] S. Dey, A. Dutta, J. I. Toledo, S. K. Ghosh, J. Lladós, and U. Pal, "SigNet: Convolutional Siamese network for writer independent offline signature verification," 2017, *arXiv:1707.02131*.
- [36] W. Tang, G. Long, L. Liu, T. Zhou, M. Blumenstein, and J. Jiang, "Omni-scale CNNs: A simple and effective kernel size configuration for time series classification," 2020, *arXiv:2002.10061*.
- [37] D. Pedamonti, "Comparison of non-linear activation functions for deep neural networks on MNIST classification task," 2018, *arXiv:1804.02763*.
- [38] A. Gulli and S. Pal, *Deep Learning With Keras*. Birmingham, U.K.: Packt Publishing Ltd, 2017.
- [39] K. G. Kim, "Book review: Deep learning," *Healthcare Informat. Res.*, vol. 22, no. 4, p. 351, 2016.
- [40] Z. Ji, J. Li, and M. Telgarsky, "Early-stopped neural networks are consistent," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 1805–1817.
- [41] W. An, H. Wang, Y. Zhang, and Q. Dai, "Exponential decay sine wave learning rate for fast deep neural network training," in *Proc. IEEE Vis. Commun. Image Process. (VCIP)*, Dec. 2017, pp. 1–4.
- [42] M. Shorfuzzaman and M. S. Hossain, "MetaCOVID: A Siamese neural network framework with contrastive loss for n-shot diagnosis of COVID-19 patients," *Pattern Recognit.*, vol. 113, May 2021, Art. no. 107700.
- [43] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. Cambridge, MA, USA: MIT Press, 2022.
- [44] E. Garcia-Ceja, M. Riegler, P. Jakobsen, J. Tørresen, T. Nordgreen, K. J. Oedegaard, and O. B. Fasmer, "Depresjon: A motor activity database of depression episodes in unipolar and bipolar patients," in *Proc. 9th ACM Multimedia Syst. Conf.*, New York, NY, USA, Jun. 2018, pp. 472–477.
- [45] A. Chatterjee, M. W. Gerdes, A. Prinz, M. A. Riegler, and S. G. Martinez, "Semantic representation and comparative analysis of physical activity sensor observations using MOX2-5 sensor in real and synthetic datasets: A proof-of-concept-study," *Sci. Rep.*, vol. 14, no. 1, p. 4634, Feb. 2024.
- [46] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, Nov. 2017.
- [47] *Medium: Supervised Anomaly Detector Models*. Accessed: Jun. 30, 2024. [Online]. Available: <https://towardsdatascience.com/supervised-machine-learning-technique-for-anomaly-detection-logistic-regression-97fc7a9cadc4>
- [48] A. Graves and A. Graves, "Long short-term memory," in *Supervised Sequence Labelling With Recurrent Neural Networks*, vol. 385. Berlin, Germany: Springer, 2012. [Online]. Available: https://doi.org/10.1007/978-3-642-24797-2_4
- [49] A. Chatterjee, M. W. Gerdes, and S. G. Martinez, "Statistical explorations and univariate timeseries analysis on COVID-19 datasets to understand the trend of disease spreading and death," *Sensors*, vol. 20, no. 11, p. 3089, May 2020.
- [50] A. Chatterjee, A. Prinz, M. A. Riegler, and Y. K. Meena, "An automatic and personalized recommendation modelling in activity eCoaching with deep learning and ontology," *Sci. Rep.*, vol. 13, no. 1, p. 10182, Jun. 2023.
- [51] A. Chatterjee, A. Prinz, and M. Riegler, "Prediction modeling in activity eCoaching for tailored recommendation generation: A conceptualization," in *Proc. IEEE Int. Symp. Med. Meas. Appl. (MeMeA)*, Jun. 2022, pp. 1–6.
- [52] A. Chatterjee, N. Pahari, A. Prinz, and M. Riegler, "Actiecoach: Personalized recommendation generation in activity ecoaching with meta-heuristic approach," *Res. Square*, Jul. 2023. [Online]. Available: <https://www.researchsquare.com/article/rs-2152305/v1> and <https://doi.org/10.21203/rs.3.rs-2152305/v1>
- [53] A. Sagheer and M. Koth, "Unsupervised pre-training of a deep LSTM-based stacked autoencoder for multivariate time series forecasting problems," *Sci. Rep.*, vol. 9, no. 1, p. 19038, Dec. 2019.
- [54] A. Chatterjee, M. W. Gerdes, and S. G. Martinez, "Identification of risk factors associated with obesity and overweight—A machine learning overview," *Sensors*, vol. 20, no. 9, p. 2734, 2020.
- [55] A. Chatterjee, N. Pahari, A. Prinz, and M. Riegler, "Machine learning and ontology in eCoaching for personalized activity level monitoring and recommendation generation," *Sci. Rep.*, vol. 12, no. 1, p. 19825, Nov. 2022.
- [56] J. Karvanen, "The statistical basis of laboratory data normalization," *Drug Inf. J.*, vol. 37, no. 1, pp. 101–107, Jan. 2003.
- [57] J. Lesouple, C. Baudoin, M. Spigai, and J.-Y. Tournet, "Generalized isolation forest for anomaly detection," *Pattern Recognit. Lett.*, vol. 149, pp. 109–119, Sep. 2021.
- [58] S. Ying, B. Wang, L. Wang, Q. Li, Y. Zhao, J. Shang, H. Huang, G. Cheng, Z. Yang, and J. Geng, "An improved KNN-based efficient log anomaly detection method with automatically labeled samples," *ACM Trans. Knowl. Discovery From Data*, vol. 15, no. 3, pp. 1–22, Jun. 2021.
- [59] M. Hosseinzadeh, A. M. Rahmani, B. Vo, M. Bidaki, M. Masdari, and M. Zangakani, "Improving security using SVM-based anomaly detection: Issues and challenges," *Soft Comput.*, vol. 25, no. 4, pp. 3195–3223, Feb. 2021.
- [60] R. Primartha and B. A. Tama, "Anomaly detection using random forest: A performance revisited," in *Proc. Int. Conf. Data Softw. Eng. (ICoDSE)*, Nov. 2017, pp. 1–6.
- [61] M. Vishwakarma and N. Kesswani, "A new two-phase intrusion detection system with Naïve Bayes machine learning for data classification and elliptic envelop method for anomaly detection," *Decis. Anal. J.*, vol. 7, Jun. 2023, Art. no. 100233.
- [62] A. P. Muniyandi, R. Rajeswari, and R. Rajaram, "Network anomaly detection by cascading K-Means clustering and C4.5 decision tree algorithm," *Proc. Eng.*, vol. 30, pp. 174–182, Jan. 2012.
- [63] T. T. Teoh, G. Chiew, E. J. Franco, P. C. Ng, M. P. Benjamin, and Y. J. Goh, "Anomaly detection in cyber security attacks on networks using MLP deep learning," in *Proc. Int. Conf. Smart Comput. Electron. Enterprise (ICSCEE)*, Jul. 2018, pp. 1–5.
- [64] M. H. Mozaffari and T. Li-Lin, "Anomaly detection using 1D convolutional neural networks for surface enhanced Raman scattering," *Proc. SPIE*, vol. 11525, pp. 162–168, Nov. 2020.
- [65] D. S. Hochba, Ed., "Approximation algorithms for NP-hard problems," *ACM Sigact News*, vol. 28, no. 2, pp. 40–52, 1997.

- [66] K. Choi, J. Yi, C. Park, and S. Yoon, "Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines," *IEEE Access*, vol. 9, pp. 120043–120065, 2021.
- [67] X. Wang, S. Yang, J. Zhang, M. Wang, J. Zhang, W. Yang, J. Huang, and X. Han, "Transformer-based unsupervised contrastive learning for histopathological image classification," *Med. Image Anal.*, vol. 81, Oct. 2022, Art. no. 102559.
- [68] M. Braei and S. Wagner, "Anomaly detection in univariate time-series: A survey on the state-of-the-art," 2020, *arXiv:2004.00433*.
- [69] S. Ahmed, I. E. Nielsen, A. Tripathi, S. Siddiqui, R. P. Ramachandran, and G. Rasool, "Transformers in time-series analysis: A tutorial," *Circuits, Syst., Signal Process.*, vol. 42, no. 12, pp. 7433–7466, Dec. 2023.
- [70] B. Tang and D. S. Matteson, "Probabilistic transformer for time series analysis," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 23592–23608.
- [71] Z. Yang, Y. Li, and G. Zhou, "TS-GAN: Time-series GAN for sensor-based health data augmentation," *ACM Trans. Comput. Healthcare*, vol. 4, no. 2, pp. 1–21, Apr. 2023.
- [72] C.-K. Lee, Y.-J. Cheon, and W.-Y. Hwang, "Studies on the GAN-based anomaly detection methods for the time series data," *IEEE Access*, vol. 9, pp. 73201–73215, 2021.
- [73] A. Chatterjee, N. Pahari, A. Prinz, and M. Riegler, "AI and semantic ontology for personalized activity eCoaching in healthy lifestyle recommendations: A meta-heuristic approach," *BMC Med. Informat. Decis. Making*, vol. 23, no. 1, p. 278, Dec. 2023.



AYAN CHATTERJEE (Member, IEEE) received the B.Eng. degree in computer science and engineering (CSE) from West Bengal University of Technology, India, in 2009, the master's degree in information technology from Jadavpur University, India, in 2016, and the Ph.D. degree from the University of Agder, Norway, in 2023. He was a Senior Researcher in AI and semantics with the Simula Research Laboratory (SimulaMet), Oslo, Norway. He is currently a Scientist with the Department of Digital Technology, STIFTELSEN NILU, and an Adjunct Associate Professor of information technology with Kristiania University College, Oslo. He has a strong aptitude for object-oriented programming concepts. His research interests include data science, knowledge engineering, human-centered AI, software engineering, and data structure.



VAJIRA THAMBAWITA (Member, IEEE) received the Ph.D. degree from Oslo Metropolitan University, Oslo, Norway, in 2021. He is currently a Senior Research Scientist with the Simula Metropolitan Center for Digital Engineering (SimulaMet), Oslo, and an Adjunct Associate Professor with Oslo Metropolitan University. His research interests include machine learning, video analysis and understanding, image processing, image retrieval, generative AI, and applications of artificial intelligence in healthcare.



MICHAEL A. RIEGLER (Member, IEEE) received the Ph.D. degree from the University of Oslo, Oslo, Norway, in 2017. He is currently the Chief Research Scientist with the Simula Metropolitan Center for Digital Engineering (SimulaMet), Oslo, and a Professor with Oslo Metropolitan University, Oslo. His research interests include machine learning, video and image analysis and understanding, image processing, image retrieval, crowd-sourcing, social computing, and biological and medical applications of machine learning.



PÅL HALVORSEN (Member, IEEE) received the Ph.D. degree from the University of Oslo, Norway, in 2001. He is currently the Chief Research Scientist with the Simula Metropolitan Center for Digital Engineering (SimulaMet), Norway; a Professor with Oslo Metropolitan University, Norway; and a Professor II with the University of Oslo. At SimulaMet, he is leading the Department of Holistic Systems Research, which investigates the challenges of complete end-to-end pipelines with a particular focus on sports and medical applications. His current research interests include several areas in distributed (multimedia) systems and content analysis from a performance and efficiency point of view. More information can be found at (<http://home.simula.no/paalh>).

...